

第二十五屆旺宏科學獎

創意說明書

參賽編號：SA25-721

作品名稱：造星者之鑰

關鍵字：物理約束學習、生成對抗網路、深空影像重建

摘要

本研究針對深度學習技術應用於深空影像重建時，在極低信噪比（SNR）環境下因過度擬合所產生之「幻覺偽影」及物理能量失真等核心問題，提出相應解決方案。現有多數純視覺導向模型雖能提升影像解析度，卻可能生成缺乏物理依據之假結構，甚至違背光通量守恆原則，影響天文觀測之科學真實性。

為改善上述問題，本研究提出融合天文物理先驗知識之生成對抗網路（Physics-Informed Conditional GAN, PI-cGAN），以 cGAN 為基礎結合物理損失函數，並使用 NASA 公開 FITS 天文影像資料建構訓練資料集，透過多版本模型迭代優化物理約束策略。研究以 PSNR、SSIM、Phase Consistency Error 與 Linear Flux Error 等指標進行模型評估與參數調整。

實驗結果顯示，相較於傳統 Basic cGAN，本研究提出之 PI-cGAN 可有效抑制幻覺偽影，並將光通量誤差由平均 214.66% 降低至 4.65%，在提升影像細節復原能力的同時，兼顧物理一致性與科學合理性，提供一套具備工程應用潛力之深空影像重建方法。

壹、研究動機

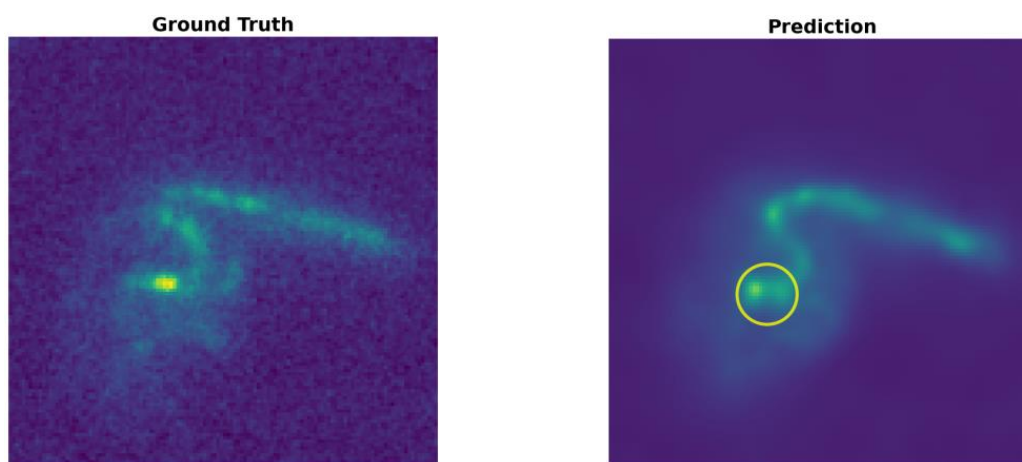
在某次瀏覽 NASA 的公開資料時，意外的發現，一般熟知的璀璨星雲與壯麗行星，並非望遠鏡直接拍攝的結果，而是經過一連串繁複的影像處理，才能得到普羅大眾所看到的星系樣貌，這件事引起了我的研究興趣（圖 1），同時也很好奇，為什麼人類所看到的深空影像必須透過影像處理輔助才能看到，這些來自宇宙遙遠的星空訊息，在紀錄與截取過程中，是不是存在著一些無法抗力的物理因素或環境限制？



處理前後之深空影像圖片（圖 1）[16]

經翻閱文獻後才了解，真實的深空觀測，受限於光學設備的點擴散函數（PSF）以及宇宙射線與泊松雜訊的干擾，影像往往被蒙上一層模糊且充滿噪點的面紗，導致信噪比（SNR）極低，換言之，目前大多的深空攝影，不只是「拍攝」，更是一場從低信噪比資料中萃取真實訊息過程的挑戰。

為了解開這層面紗，先從目前可行方法的做改進，受到近年深度學習模型的快速發展影響（如生成對抗網路 cGAN[1]與 U-Net 架構[2]），發現在 2017 年曾有人使用過 cGAN 進行了深空影像還原[3]，但是基於當時為 cGAN 模型發展之初期，文章當中提及的方法雖能有效提升 PSNR 指標（與傳統方法相比），卻不能驗證其模型產出之星體位置是正確的，如同[4]中提及之深度學習模型在天文影像處理時產生的「幻覺偽影」問題（圖 2-1、圖 2-2）有高度關聯性。經由初步的數次測試，確實發現了此問題：傳統純視覺驅動的 cGAN 模型雖然能有效提升影像的視覺清晰度，但為了一昧追求肉眼看似清晰的對比度，AI 的輔助反而變成了幻覺製造者，在原本充滿雜訊的太空影像裡，生成出一些不符合物理意義的假結構。



正常的圖(圖 2-1)

出現幻影的圖(圖 2-2)

圖片來源：[4] (Figure 1.)

因此開始思考：能否在基礎的 cGAN 架構中寫入宇宙的物理定律，讓它不再那麼隨意的生成不符合物理條件之影像？於是開始反向思考，能不能將物理條件納入機器學習的一部份，經多方的嘗試與調整，重新發展一套融合了頻域特徵分析的物理約束條件式生成對抗網路（PI-cGAN），不再依賴純粹的視覺數據驅動，而是同時將高頻異常特徵與光通量守恆公式引入，並使其成為主架構中的第二個數據驅動，為其加上更符合實際情況的物理環境。期望在提升深空影像解析度的同時，亦能抑制深度學習的幻覺現象。

貳、研究目的

解決空間數據深度學習模型於深空影像還原中易產生的「幻覺偽影」之問題，並建立兼具影像品質與物理準確性的 cGAN 架構。具體研究指標如下：

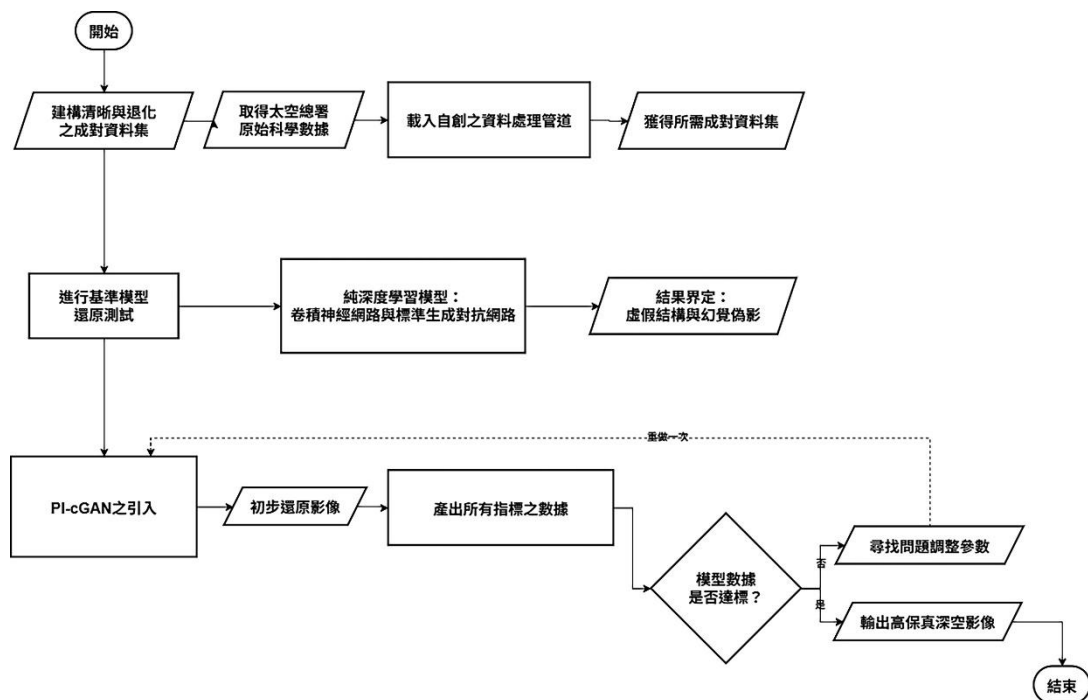
- 一、建立具物理真實性之天文影像成對資料集生成流程
- 二、驗證純視覺驅動 cGAN 是否存在偽影與光通量失真問題
- 三、建立出一套能有效抑制幻覺偽影並提升物理一致性之深度學習模型

參、研究過程與方法

本節回顧相關文獻與理論基礎，並說明本研究之資料建構流程與模型訓練方法，主要分為以下四個部分：

1. NASA FITS 圖檔資料預處理管道。
2. 基準模型（Baseline）之問題定義與設定。
3. PI-cGAN 架構於 V1 至 V5 之模型設計與訓練策略演進。
4. 各模型之實驗結果與數據比較分析。

進入實驗後之具體流程圖如下所示：



3.1 文獻探討與學理基礎

考量主體核心問題在處理深空影像與機器訓練的改進，為確保本模型架構之學理嚴謹性，因此以「天文物理學」與「深度學習」兩大領域進行回顧：

- 不適定反問題與深空影像退化機制：Lagendijk 等人（1988）[7]指出，影像還原問題在數學上屬於典型的「不適定反問題（ill-posed inverse problems）」，其特性為解的不唯一性與對雜訊高度敏感性。於深空影像中，此一不穩定性主要來自於觀測系統的物理退化過程。首先，由於光學成像系統受到繞射極限影響，原始天文訊號會與點擴散函數（Point Spread Function, PSF）進行卷積，使影像產生不可避免的模糊現象。其次，在感測器進行光子計數的過程中，會同時受到 Foi 等人（2008）[8]所描述之 Poisson–Gaussian 混合雜訊干擾，使影像信噪比（SNR）進一步下降。基於上述兩種主要退化來源，深空影像復原問題本質上即為同時處理模糊與隨機雜訊的反問題。因此，在後續資料處理管道中，針對 NASA FITS 影像建立相應的退化與還原模組，以模擬真實觀測條件。此外，針對宇宙天體的高動態範圍特性，本研究亦參考 Lupton 等人（2004）[5]所提出之影像重建方法，以改善亮度分布在視覺化與數據分析間的平衡問題，採用反雙曲正弦（Asinh）函數進行影像拉伸，以有效保留恆星與微弱星雲細節。
- 深度學習幻覺與感知失真兩難：近年 U-Net 等模型廣泛應用於天文影像，然而，Gottschling 等人（2020）[10]於《PNAS》指出，純數據驅動模型在處理不適定反問題時，極易產生「幻覺偽影(Hallucination Artifacts)」。最新研究 CHEM（2025）[4]針對專門處理天文影像的深度模型進一步調查，證實了 AI 為迎合視覺清晰度，常不經意的產生不具物理意義的結構，深刻印證了影像重建中著名的「感知與失真兩難」[12]，亦即如果僅追求單一指標，模型時常會為了過度追求視覺上的精美，而生成許多不符合現實物理中的結構。。
- 物理約束神經網路 (PINNs) 與頻域分析：為抑制 AI 失控，Raissi 等人（2019）[6]提出「物理約束神經網路（PINNs）」，倡導將物理先驗知識寫入損失函數。同時 Frank 等人（2020）[9]指出虛假影像在頻域會暴露出異常能量分佈。本研究初期結合上述學理[4][5][6][7][8][10][12]，從開發具有退化模塊的資料預處理管道，再到主要的模型訓練，加入了快速傅立葉轉換（FFT）與線性光通量守恆公式，後續驗證除了使用傳統的 PSNR 與 SSIM 指標，也引入了 Phase Consistency

Error([15]得來之靈感) 與 Linear Flux Error(由[14]得來之靈感)兩個物理的指標來確保模型在物理定義上的精確性，建構專為深空影像還原設計的 PI-cGAN 架構。

3.2 具備物理意義之資料建構管道

為確保深空影像學習過程建立於合理的物理退化假設，本研究使用 NASA 公開之 FITS 深空影像資料，並設計 AstroDegrader 模組以模擬天文觀測中的成像退化流程。

1. 影像切片與資料前處理：

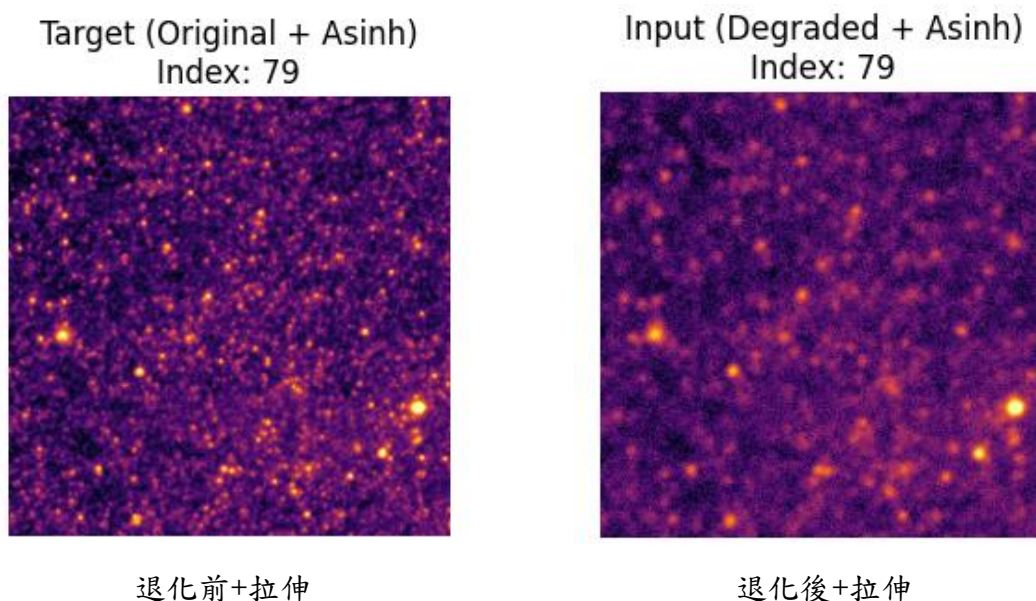
為提升訓練效率並降低記憶體負擔，將原始 FITS 影像以滑動視窗方式切割為 256×256 像素之局部區塊（重疊步長 128）。同時排除無有效訊號或純背景之區塊，並在完成切片後釋放原始影像以優化記憶體使用。

2. 線性物理域退化建模：

在保持線性光子統計假設的條件下，對影像切片進行前向退化處理。首先使用 Moffat 2D 核函數模擬點擴散函數（PSF）造成之光學模糊效應，隨後疊加 Poisson–Gaussian 混合雜訊，同時反映光子統計噪聲與感測器讀出噪聲特性。

3. 非線性拉伸轉換與數值正規化：

退化完成後，對影像進行基於分位數（0.1%–99.9%）之截斷處理，以降低極端值（如宇宙射線）對分布的影響。隨後採用 asinh（inverse hyperbolic sine）非線性轉換（參數 $a = 0.1$ ）對真實影像與退化影像進行一致性壓縮，並將數值正規化至 $[-0.05, 1.05]$ 範圍，以符合神經網路輸入尺度需求。



- #### 4. 控制變因：
- 資料處理後，在模型訓練時統一使用固定亂數種子（Seed=42）切分 80% 訓練集與 20% 測試集，以確保不同模型版本之間具有一致比較基準。

3.3 評估機制：檢測腳本與四大核心指標

為客觀比較各版本模型之重建表現，本研究依據 3.1 文獻探討內容與初步實驗結果，設定四項核心評估指標與合理目標區間如下：

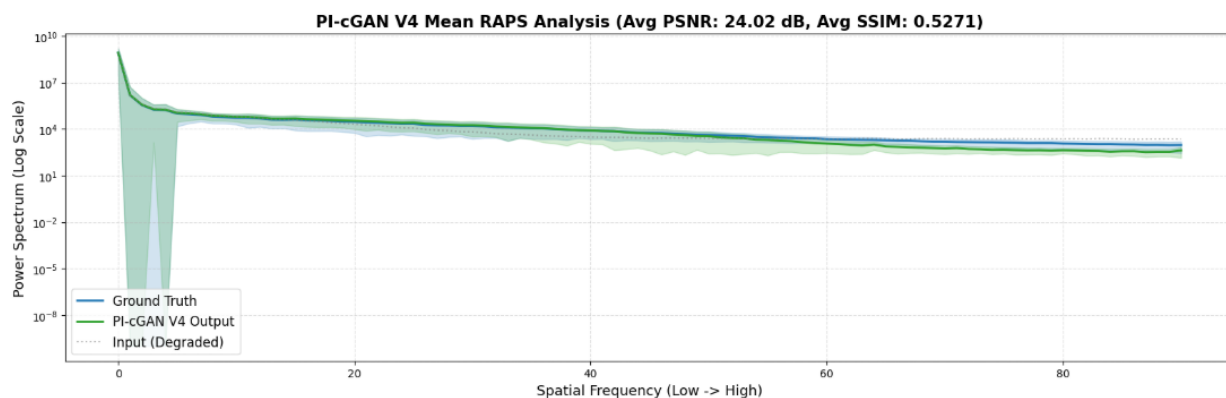
1. **峰值信噪比 (PSNR)**：目標設定為 > 30 dB，評估影像整體訊噪抑制與像素重建品質。
2. **結構相似度 (SSIM)**：考量深空影像於極低信噪比環境下之高度退化特性，目標設定為 > 0.65 ，以評估天體輪廓與結構還原能力。
3. **相位一致性誤差 (Phase Consistency Error, rad)**：目標設定為 < 1.5 ，用以衡量高頻特徵於頻域中的相位一致性與結構穩定性。
4. **線性光通量誤差率 (Linear Flux Error %)**：指標目標設定為 $< 5\%$ ，以確保經還原後的影像能維持在天文物理學中容許的光通量誤差之門檻內。

模型訓練完成後，皆透過自行開發之檢測腳本自動化分析生成結果，輸出各項數據並繪製比較圖表，以檢驗模型是否達成上述目標並作為版本迭代優化依據。

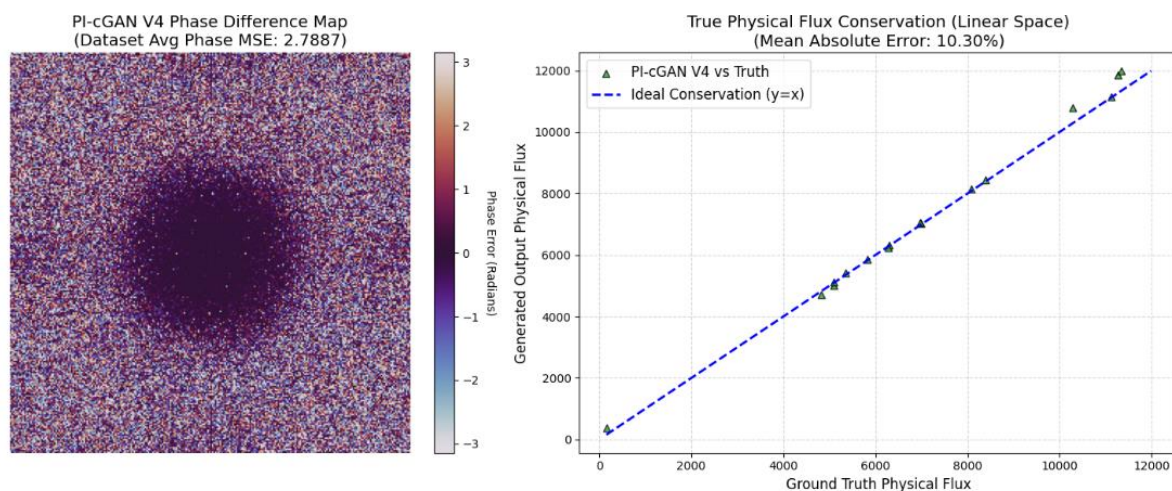
備註：前期版本模型 (V1 至 V3) 之最優權重篩選機制

由於前期的 Baseline、V1、V2 與 V3 版本模型皆採用固定 200 個 Epoch 的訓練設定，且在訓練過程中採取每 40 個 Epoch 定期保存一次權重的策略。為了精確找出各模型訓練過程中之最優權重，本研究在執行最終評估前，額外加入了掃描腳本。該腳本會測試所有存取之權重，並給與各權重之數據；經由我們判斷並篩選出最佳模型權重後，方正式導入「評估腳本」產出最終之量化報表與科學分析數據。

下表以 PI-cGAN V4 模型產出之圖表為例：



PSNR、SSIM 指標圖表

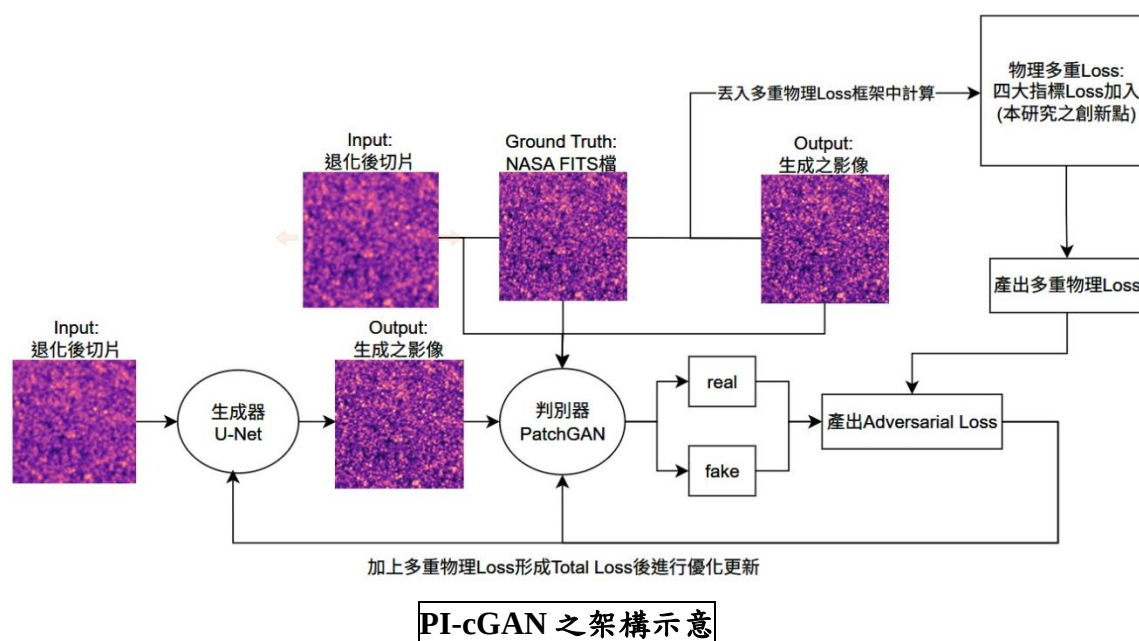
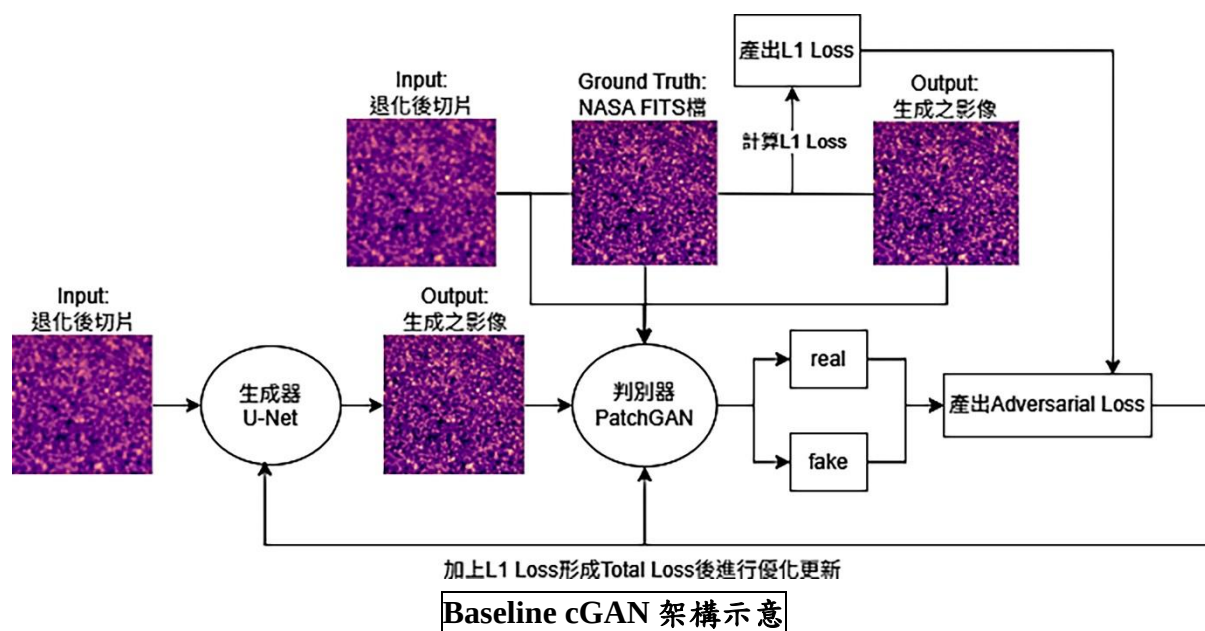


Phase Consistency error、Linear Flux error 指標圖表

由上述圖表可知各項數據是否達標，並進行下一步的規劃

3.4 cGAN 結構以及創新部位

在進入模型訓練前，我們可以從[1]了解到 cGAN 的架構，這裡將用明確的圖來說明後續階段性成果中所提及的 baseline cGAN 與我們自行開發之 PI-cGAN 的明確結構差異，由圖中可以明確知道兩者的差異位於圖中右上角的 Loss，PI-cGAN 多了物理多重 Loss 而非只有 L1 Loss，這對應了我們想加入物理約束的目標。



3.5 階段性成果一：基準模型界定與初期嘗試 (Baseline & V1)

3.5.1 基準模型：採用純視覺驅動架構，無物理約束。

損失函數：

$$L_{total} = L_{GAN} + 100L_{L1}$$

其中， L_{GAN} 代表對抗損失， L_{L1} 代表像素級損失

評估結果：將結果逆推回線性物理空間後，測試集之平均光通量誤差高達 214.66%，測試集之平均光通量誤差高達 214.66%，個別樣本誤差最高甚至超過 1800%，這顯示模型未能維持真實光子分布。

3.5.2 PI-cGAN V1 (初始約束嘗試)：為了降低偽影現象，我們首次於 Loss Function 中加入 FFT Loss 與 Flux Loss：

損失函數：

$$L_{total} = L_{GAN} + 100L_{L1} + 10L_{FFT} + 5L_{Flux}$$

其中， L_{FFT} 代表頻域損失， L_{Flux} 代表光通量損失

評估結果：然而第一次實驗結果並未改善，反而讓 Flux Error 上升至 226.94% (平均誤差)。

後續重新檢查資料流程時，我們發現問題可能來自於 Asinh 非線性空間。由於 Asinh 會改變原始光子比例，因此直接在該空間中計算物理量，可能導致模型學到錯誤能量分布。

3.6 階段性成果二：核心突破與優化飽和瓶頸 (V2 ~ V3)

3.6.1 PI-cGAN V2 (克服梯度衝突)：V1 雖成功加入物理概念，但過強的物理限制會導致模型早期訓練不穩定。因此 V2 導入「課程學習 (warmup_epochs=100)」，並下調物理權重，讓模型先學習空間結構，再逐步加入物理限制。V2 同時也加入 GT 星體遮罩，降低梯度衝突問題。

損失函數：

$$L_{total} = L_{GAN} + 100L_{L1} + 0.1L_{FFT} + 0.01L_{Flux}$$

評估結果：此版本成功將光通量誤差降低至 7.17%。

3.6.2 PI-cGAN V3 (線性空間逆映射極限)：繼承前面模型的經驗，目前的核心突破，已可將 **Linear Flux Error** 有效的降低至平均不到 5%。模型於計算能量損失前，我們嘗試將生成結果先透過 `inverse_to_linear` 函數將預測結果逆推回線性物理域，再計算 Flux Loss：

$$I_{\text{linear}} = \sinh(I_{\text{asinh}} \cdot a)$$

損失函數：參數同 V2，但 L_{Flux} 改於線性空間計算。

評估結果：我們觀察到光通量誤差明顯下降，最佳權重（迴圈 120）之平均光通量誤差最低可壓低至平均 4.65%。

然而，V3 雖提升物理一致性，SSIM 反而停滯於 0.4636。我們推測，加入的物理限制可能抑制了部分高頻細節生成，因此開始出現感知品質與物理正確性之間的競爭現象。

3.7 後續測試：後續研究規劃與最終測試 (V4 ~ V5)

為突破 V3 遭遇之「感知與失真兩難」，目前正在進行下一階段之典範轉移與微調，預計透過以下策略產出最終解：

3.7.1 策略一：PI-cGAN V4 (載入原始權重)：不再從零訓練，而是載入 V3 最佳權重（Epoch 120）為起跑點，學習率由 2×10^{-4} 降至 2×10^{-5} ，並於空間損失中全新導入多尺度結構相似度（MS-SSIM），同時開始設置上限迴圈數(模型未持續優化時使其停止迴圈並儲存權重和結果)10 圈與總迴圈數 50 圈。

損失函數設計：

$$L_{\text{total}} = L_{\text{GAN}} + 100L_{\text{MS-SSIM}+L_1} + 0.1L_{\text{FFT}} + 0.01L_{\text{Flux_Linear}}$$

初步進展：結果顯示，有潛力將 SSIM 躍升至 0.5271，同時維持約 10.30% 之低誤差。

3.7.2 策略二：PI-cGAN V5 (多種類物理約束模型)：V5 為目前測試中的多重物理限制架構，進一步加入：FFT Amplitude、Phase Loss、MSE Loss，且與 V4 相同存在著限制迴圈(V5 為 15 個，增加其容錯率因為損失函數多)以及總迴圈數 50 圈。

損失函數設計：

$$L_{\text{total}} = L_{\text{GAN}} + 100L_{L_1} + 5L_{\text{FFT_Amp}} + 20L_{\text{Phase}} + 100L_{\text{Flux_Linear}}$$

初步進展：初步測試發現，當多種物理限制同時存在時，模型容易出現新的梯度競爭問題，觀察到 Flux Error 反而回升至 13.14%。我們因此推測，深度學習模型在多目標優化下，可能存在可承受之物理限制上限。

3.7.3 策略三：PI-cGAN V4.1 (V4 參數微調+長訓練週期)：由於發現 V5 可能存在著物理限制的上限，所以我們重新回到了獲得不錯結果的 V4 並修改其參數及迴圈數，微調 L_{Flux} 權重至 0.5，試圖在提升畫質的同時進一步維持物理正確性，V4.1 存在著更多的限制迴圈數(20 圈)以及總迴圈數 100 圈，在初期時表現穩定且跑完了總迴圈數依然沒有摸到限制迴圈數的門檻，於是我們重新載入了 V4.1 第一階段的權重用且用同樣的參數繼續跑，並將限制迴圈設置到 30 圈與總迴圈數 200 圈，讓其繼續運轉。

損失函數設計：

$$L_{total} = L_{GAN} + 100L_{MS-SSIM+L1} + 0.1L_{FFT} + 0.5L_{Flux_Linear}$$

實驗發現：即便調高 Flux 權重至 0.5 並進行第二階段訓練，其平均 Flux Error 仍略升至 10.54%，且 SSIM 下滑至 0.4921。

3.8 PI-cGAN 模型演進路徑與物理約束邏輯分析

3.8.1 演進階段分析

- **探索期 (V1)：**新增物理約束，研究了解：非線性空間的物理量無意義。
- **突破期 (V2-V3)：**課程學習與線性逆映射，確立能量守恆極限。
- **優化期 (V4-V5)：**多方面改善，研究了解：梯度衝突與參數細部調整之差異。

3.8.2 核心學理發現：線性空間的必要性

從實驗結果得知：直接於 Asinh 拉伸空間計算 Flux 損失會導致模型產生負向引導（如 V1 之 Linear Flux Error 平均高達 226.94%）。有別於傳統研究多在非線性拉伸空間進行特徵逼近，本研究使用了「線性空間逆映射技術」。透過 $\sinh$ 函數將生成網路輸出逆推回原始光子域進行守恆約束，這是將誤差由 226.94% 壓低至 4.65% 的關鍵技術分水嶺。

3.8.3 物理承載上限與「反彈邊界」

在 V5 的開發中，本研究觀察到一個重要現象：當損失函數同時包含超過四種物理約束時，模型會因**梯度競爭**出現收斂震盪。這暗示了深度學習模型在特定參數規模下，存在其『物理約束承載上限』，並非約束越多越好。」

3.9 階段性量化對比

彙整本研究目前各模型世代產出之綜合科學指標數據：

模型版本	核心技術與訓練特徵	PSNR	SSIM	Phase Consistency Error	Linear Flux Error
Baseline	L1 + GAN (純視覺驅動)	22.1709	0.4654	2.8174 rad	214.66%
V1	非線性空間計算 Flux/FFT	22.4168	0.4688	2.8273 rad	226.94%
V2	課程學習 + GT Mask	22.3834	0.4717	2.8052 rad	7.17%
V3	線性空間逆映射	22.6964	0.4636	2.8377 rad	4.65%
V4	V3 權重加入+ MS-SSIM	24.0194	0.5271	2.7887 rad	10.30%
V4.1	基於 V4 更改 Flux 權重	22.6105	0.4921	2.7845 rad	10.54%
V5	多種類物理約束模型	23.7870	0.5288	2.7923 rad	13.14%

(測試集包含 16 張獨立 FITS 影像切片，所有評估數據皆為測試樣本平均值，並統一以小數點後第四位進行四捨五入表示。)

肆、研究成果與未來展望

本研究前期透過多次實驗驗證發現，傳統純視覺導向之深度學習模型於深空影像重建中易產生「幻覺偽影」現象。在未進行物理約束修正之情況下，其光通量一致性評估中觀察到平均高達 214.66% 之相對誤差，顯示模型於極低信噪比條件下存在顯著物理失真問題。

為改善上述問題，本研究提出 PI-cGAN 架構，並逐步進行模型改良，引入線性空間逆映射與物理約束條件。實驗結果顯示，PI-cGAN V3 模型可將光通量誤差降低至平均 4.65%，並在影像品質與物理一致性之間取得較佳平衡。

PI-cGAN V4 版本目前仍處於優化與測試階段，後續將持續調整模型結構與物理約束設計，以提升模型於極低信噪比環境下之泛化能力與可靠性。

在 PI-cGAN V5 初步實驗中，雖引入更強之物理約束設計，但觀察到光通量誤差反而上升，推測可能與約束條件過強導致模型學習受限或優化不穩定有關。因此後續將回歸 V4 架構進行調整，並發展 V4.1 版本重新配置物理約束權重，以進一步提升模型整體表現。

基於自行開發之檢測腳本與四大核心評估指標（PSNR、SSIM、Phase Consistency Error、Linear Flux Error），本研究已建立一套可量化分析模型重建品質與物理一致性之驗證流程。現階段雖已取得初步階段性成果，整體模型表現仍未全面達成預設目標區間。後續將持續優化模型結構、損失函數配置與物理約束權重，分析不同參數設定對模型穩定性、泛化能力及物理一致性之影響，逐步提升深空影像重建之物理可信度與整體效能。

伍、參考文獻

- [1] M. Mirza and S. Osindero, “Conditional generative adversarial nets,” *arXiv preprint arXiv:1411.1784*, 2014.
- [2] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, Springer, 2015.
- [3] K. Schawinski, P. Zhang, H. Zhang, J. Fowler, G. Santhanam, and A. Lee, “Generative adversarial networks recover features in astrophysical images of galaxies beyond the deconvolution limit,” *Monthly Notices of the Royal Astronomical Society*, vol. 467, no. 4, pp. L110–L114, 2017.
- [4] J. Li, I. Rosellon-Inclan, G. Kutyniok, and J.-L. Starck, “CHEM: Estimating and understanding hallucinations in deep learning for image processing,” *arXiv preprint arXiv:2512.09806*, 2025.
- [5] R. Lupton et al., “Preparing red-green-blue images from CCD data,” *Publications of the Astronomical Society of the Pacific*, 2004.
- [6] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.
- [7] A. K. Katsaggelos, “A regularized iterative image restoration algorithm,” *IEEE Transactions on Signal Processing*, vol. 39, no. 4, pp. 471–482, Apr. 1991.
- [8] A. Foi, M. Trimeche, V. Katkovnik, and K. Egiazarian, “Practical Poissonian-Gaussian noise modeling and fitting for single-image raw-data,” *IEEE Transactions on Image Processing*, vol. 17, no. 10, pp. 1737–1754, 2008.
- [9] J. Frank, T. Eisenhofer, L. Schöenherr, A. Fischer, D. Kolossa, and T. Holz, “Leveraging frequency analysis for deep fake image recognition,” in *International Conference on Machine Learning (ICML)*, 2020.
- [10] N. M. Gottschling, V. Antun, A. C. Hansen, and B. Adcock, “The troublesome kernel: On hallucinations, no free lunches and the accuracy-stability trade-off in inverse problems,” *Proceedings of the National Academy of Sciences (PNAS)*, 2020.
- [11] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, “Curriculum learning,” in *Proceedings of the 26th International Conference on Machine Learning (ICML)*, 2009.
- [12] Y. Blau and T. Michaeli, “The perception-distortion tradeoff,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

- [13] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [14] N. Padmanabhan et al., “An improved photometric calibration of the Sloan Digital Sky Survey imaging data,” Lawrence Berkeley National Laboratory Report LBNL-61732, 2006..
- [15] A. V. Oppenheim, J. S. Lim, and S. R. Curtis, “The importance of phase in signals,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 29, no. 3, pp. 529–541, 1981.
- [16] National Aeronautics and Space Administration (NASA), “Webb telescope image processing overview,” *NASA Science*, [Online]. Available: <https://science.nasa.gov/>. [Accessed: 19-May-2026].
- [17] European Space Agency, “How telescopes capture deep field images,” *ESA*, [Online]. Available: <https://www.esa.int/>. [Accessed: 19-May-2026].

陸、附錄

1. 資料集來源與科學代表性

本研究之所有訓練與測試數據，均源自 NASA 密爾基天文資料庫（MAST, Mikulski Archive for Space Telescopes）公開之 FITS（Flexible Image Transport System）天文影像圖檔。在樣本選擇上，本研究挑選了 **M31（仙女座星系）**、**M51（漩渦星系）** 以及 **M101（風車星系）** 作為核心訓練來源。此三者在天文物理形態學上具備一定程度之代表性，包含了豐富的懸臂結構、緻密核心、恆星形成區以及多變的氣體塵埃分佈，能有效考驗模型在不同宇宙結構下的還原能力。

為確保輸入影像的物理一致性並排除不必要的干擾變因，固定使用了一套標準化的數據篩選流程。所有擷取的 FITS 圖檔皆限制於以下檢索條件：

- 任務與儀器（Mission & Instrument）：設定為哈伯太空望遠鏡（HST）的先進巡天照相機寬視野通道（ACS/WFC）。其高空間解析度能提供精細的次角秒（sub-arcsecond）天體結構。
- 波段濾鏡（Filters）：統一選用 F814W 濾鏡（相當於近紅外線的 I 波段）。該波段對老年的恆星群與星系骨架結構較為敏感，且受星際塵埃消光的影響較小，利於模型捕捉清晰的形態特徵。
- 資料類型與校準等級（Type & Calibration Level）：篩選類型為科學觀測影像（Science Product），且校準等級必須達到 Level 3。這代表影像已通過 MAST 的標準管線處理（包含偏壓、平場修正以及宇宙線剔除），大幅降低了儀器雜訊對模型學習物理約束時的干擾。

透過上述之篩選條件，本研究得以精準地建構出具備高信噪比與科學研究價值的影像數據集(經過資料預處理管道，產生出了 7133 組成對訓練資料，並在每次訓練前固定使用亂數種子（Seed=42）切分 80% 訓練集與 20% 測試集去做使用)，為後續的影像恢復模型奠定了堅實的數據基礎。

2. 評估指標定義與目標設定介紹

本研究處理之資料為科學級天文數據，單一的電腦視覺（CV）評估指標無法完全涵蓋天文觀測所需之嚴謹性。為確保 PI-cGAN 模型生成的影像在形態學與測光學上具備高度可靠性，前面文章所設定的四大指標皆有明確的定義以及參考：

1. 峰值信噪比 (Peak Signal-to-Noise Ratio, PSNR)

- 目標設定：> 30 dB
- 指標意義：PSNR 衡量生成影像與真實影像在像素層級的絕對誤差。

- 數學公式：

$$\text{MSE} = \frac{1}{mn} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} [I(i,j) - K(i,j)]^2$$

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{\text{MAX}_I^2}{\text{MSE}} \right)$$

(其中， MAX_I 為影像可能的最大像素值， MSE 為均方誤差。)

- 設定依據：原始天文影像中的雜訊普遍具有 Poisson–Gaussian 混合特性（Foi 等人，2008 [8]）。為避免模型在追求視覺清晰度時過度扭曲像素特徵（Blau 與 Michaeli，2018[12]），本研究將 PSNR 目標設定為 > 30 dB（即有效訊號能量達殘留雜訊之 1000 倍以上）。此門檻能在不模糊星系邊緣的前提下，確保模型在非線性去噪與真實訊號保留之間取得優良之平衡。

2. 結構相似度 (Structural Similarity Index Measure, SSIM)

- 目標設定： > 0.65
- 指標意義：SSIM 突破了傳統像素比較的限制，從亮度、對比度與結構三個維度評估影像。這對於確保仙女座（M31）或漩渦星系（M51）等複雜幾何結構的高還原度十分的重要。
- 數學公式：

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}$$

(其中， μ 為均值， σ^2 為變異數， σ_{xy} 為協方差， c_1, c_2 為穩定常數。)

- 設定依據：基於科學觀測對幾何結構變化的敏感度（Wang 等人，2004[13]），本研究引入 SSIM 指標。參考前人使用 GAN 還原深空影像時的數據（Schawinski 等人，2017[3]），將目標設定為 > 0.65 （而非自然影像常見的 0.9）。此設定已足以確保星系旋臂與棒狀核心等關鍵形態學特徵被成功保存，具備科學觀測價值。

3. 相位一致性誤差 (Phase Consistency Error)

- 目標設定： < 1.5 rad
- 指標意義：在頻域中，振幅決定了能量大小，而相位決定了特徵的空間位置，控制相位誤差是為了防範模型憑空捏造假恆星或假星際塵埃。
- 數學公式：

可定義為預測影像與真實影像在二維傅立葉轉換後的相位角絕對誤差之加權和：

$$\mathcal{L}_{\text{phase}} = \frac{1}{N} \sum_{u,v} |\angle F_{\text{pred}}(u, v) - \angle F_{\text{true}}(u, v)|$$

(註：採用弧度制，1.5 約為相角誤差上限，確保高頻邊緣不發生過度的幾何位移。)

- 設定依據：根據 Oppenheim 等人(1981)[15]之研究，相位資訊的準確性遠比振幅更為關鍵，任何相位偏移都將直接引發幾何結構的錯位，鑒於深度學習模型在求解影像逆問題時，極易產生的「幻覺偽影」與高頻頻譜偽影（Li 等人，2025[4]；Gottschling 等人，2020[10]；Frank 等人，2020[9]），本研究採用標準弧度制，並將目標定於 < 1.5 rad，此設定能透過傅立葉轉換落實頻域對齊，強制模型學習真實天體的空間頻率分佈，確保還原出的影像結構具備物理真實性。

4. 線性光通量誤差率 (Linear Flux Error %)

- 目標設定： $< 5\%$
- 指標意義：光通量代表天體發出的總光子數量，關乎測光學中的星等計算與恆星質量估計，若影像處理造成光通量流失，該影像即失去科學量測意義。
- 數學公式：

$$\text{Flux Error}(\%) = \left| \frac{\sum_{i,j} I_{\text{pred}}(i,j) - \sum_{i,j} I_{\text{true}}(i,j)}{\sum_{i,j} I_{\text{true}}(i,j)} \right| \times 100\%$$

- 設定依據：在天文測光學中，光通量的守恆是科學量測的絕對基礎（Padmanabhan 等人，2006[14]）。神經網路的非線性轉換常導致光度資訊丟失（Lupton 等人，2004[5]），因此將光通量誤差限制於 $< 5\%$ 之內（相當於測光誤差約 0.05 等），可以確保模型還原之影像維持在天文物理學容許的誤差範圍內。

以上為四大指標之定義與目標設定介紹。

3. 各版本之程式碼

這裡將會放上資料預處理管道、檢測腳本以及各版本模型之程式碼以利了解其中運作原理。

1. 資料預處理管道：

```
import os
import glob
import gc
import numpy as np
from astropy.io import fits
from skimage.util import view_as_windows
from astropy.convolution import convolve, Moffat2DKernel
from astropy.visualization import AsinhStretch

class AstroDegradar:
```

```

def __init__(self, psf_fwhm=3.5, exptime=1000.0, read_noise_std=0.01):
    self.psf_fwhm = psf_fwhm
    self.exptime = exptime
    self.read_noise_std = read_noise_std

    gamma = psf_fwhm / (2 * np.sqrt(2**(1/4.765) - 1))
    self.kernel = Moffat2DKernel(gamma=gamma, alpha=4.765)

def apply_degradation(self, clean_linear_patch):
    # 光學模糊
    blurred = convolve(clean_linear_patch, self.kernel, boundary='extend')
    # 泊松散粒雜訊
    expected_photons = np.clip(blurred * self.exptime, 0, None)
    noisy_photons = np.random.poisson(expected_photons)
    noisy_flux = noisy_photons / self.exptime
    # 高斯讀取雜訊
    read_noise = np.random.normal(0, self.read_noise_std, noisy_flux.shape)
    return noisy_flux + read_noise

def process_single_fits(fits_path, output_dir, degrader, patch_size=256, stride=128):
    base_name = os.path.splitext(os.path.basename(fits_path))[0]
    save_path = os.path.join(output_dir, f"dataset_{base_name}.npz")

    if os.path.exists(save_path):
        print(f"跳過: {base_name} (檔案已存在)")
        return

    print(f"處理: {base_name}")

    try:
        # Phase A: 讀取與切片
        with fits.open(fits_path) as hdul:
            data_ext = next((i for i, hdu in enumerate(hdul) if hdu.header.get('EXTNAME') ==
'SCI'), 0)
            image = hdul[data_ext].data

            image_clean = np.ascontiguousarray(np.nan_to_num(image, nan=0.0))
            mask_float = np.ascontiguousarray(((image != 0) & np.isfinite(image)).astype(float))

            img_patches = view_as_windows(image_clean, (patch_size, patch_size),
step=stride).reshape(-1, patch_size, patch_size)
            mask_patches = view_as_windows(mask_float, (patch_size, patch_size),
step=stride).reshape(-1, patch_size, patch_size)

```

```

target_linear_patches = [p_img for p_img, p_mask in zip(img_patches, mask_patches) if
np.min(p_mask) != 0 and np.std(p_img) > 1e-5]

del image, image_clean, mask_float, img_patches, mask_patches
gc.collect()

if not target_linear_patches:
    print(f"警告: {base_name} 沒有產生任何有效切片。")
    return

# Phase B: 退化與非線性轉換
asinh_transform = AsinhStretch(a=0.1)
input_norm_patches = []
target_norm_patches = []

for clean_linear in target_linear_patches:
    degraded_linear = degrader.apply_degradation(clean_linear)

    v_min = np.percentile(clean_linear, 0.1)
    v_max = np.percentile(clean_linear, 99.9)
    if v_max == v_min: v_max = v_min + 1e-8

    clean_norm = np.clip(asinh_transform((clean_linear - v_min) / (v_max - v_min)), -
0.05, 1.05)
    degraded_norm = np.clip(asinh_transform((degraded_linear - v_min) / (v_max -
v_min)), -0.05, 1.05)

    input_norm_patches.append(degraded_norm)
    target_norm_patches.append(clean_norm)

inputs_f32 = np.array(input_norm_patches, dtype=np.float32)
targets_f32 = np.array(target_norm_patches, dtype=np.float32)

# Phase C: 存檔與 Metadata 封裝
metadata = {'psf_fwhm': degrader.psf_fwhm, 'exptime': degrader.exptime, 'read_noise':
degrader.read_noise_std, 'asinh_a': 0.1}
np.savez_compressed(save_path, inputs=inputs_f32, targets=targets_f32,
metadata=metadata)

print(f"儲存 {inputs_f32.shape[0]} 張成對切片至 -> {save_path}")

except Exception as e:

```



```

        print(f"處理 {base_name} 時發生錯誤: {e}")

if __name__ == "__main__":
    INPUT_DIR = "raw_fits"
    OUTPUT_DIR = "processed_dataset"

    os.makedirs(OUTPUT_DIR, exist_ok=True)

    my_degrader = AstroDegrader(psf_fwhm=3.5, exptime=1000.0, read_noise_std=0.01)

    fits_files = glob.glob(os.path.join(INPUT_DIR, "*.fits"))
    print(f"找到 {len(fits_files)} 個 FITS 檔案。")

    for file_path in fits_files:
        process_single_fits(file_path, OUTPUT_DIR, my_degrader)

    print("全部 FITS 檔案處理完成。")

```

2.Baseline cGAN:

```

import os
import glob
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split
import matplotlib.pyplot as plt
from IPython.display import clear_output
from tqdm.notebook import tqdm

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"目前使用的運算裝置: {device}")

class DeepSpaceDataset(Dataset):
    def __init__(self, npz_folder):
        self.inputs = []
        self.targets = []
        file_paths = glob.glob(os.path.join(npz_folder, "*.npz"))

        print(f"載入 {len(file_paths)} 個 .npz 資料檔...")
        for f in file_paths:
            data = np.load(f)
            self.inputs.append(data['inputs'])
            self.targets.append(data['targets'])

```

```

self.inputs = np.concatenate(self.inputs, axis=0)
self.targets = np.concatenate(self.targets, axis=0)
self.inputs = np.expand_dims(self.inputs, axis=1)
self.targets = np.expand_dims(self.targets, axis=1)
print(f"資料載入完成，總切片數: {self.inputs.shape[0]}")

def __len__(self):
    return self.inputs.shape[0]

def __getitem__(self, idx):
    x = torch.tensor(self.inputs[idx], dtype=torch.float32)
    y = torch.tensor(self.targets[idx], dtype=torch.float32)
    x = torch.clamp((x - 0.5) * 2.0, -1.0, 1.0)
    y = torch.clamp((y - 0.5) * 2.0, -1.0, 1.0)
    return x, y

DATA_DIR = "processed_dataset"
full_dataset = DeepSpaceDataset(DATA_DIR)

dataset_size = len(full_dataset)
test_size = int(0.2 * dataset_size)
train_size = dataset_size - test_size

train_dataset, test_dataset = random_split(full_dataset, [train_size, test_size],
generator=torch.Generator().manual_seed(42))

train_dataloader = DataLoader(train_dataset, batch_size=16, shuffle=True, drop_last=True)
test_dataloader = DataLoader(test_dataset, batch_size=16, shuffle=False, drop_last=False)

print(f"資料集切分: 訓練集 {train_size} 筆, 測試集 {test_size} 筆")

class UNetDown(nn.Module):
    def __init__(self, in_channels, out_channels, normalize=True, dropout=0.0):
        super().__init__()
        layers = [nn.Conv2d(in_channels, out_channels, kernel_size=4, stride=2, padding=1,
bias=False)]
        if normalize:
            layers.append(nn.BatchNorm2d(out_channels))
        layers.append(nn.LeakyReLU(0.2))
        if dropout > 0.0:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)

```

```

def forward(self, x):
    return self.model(x)

class UNetUp(nn.Module):
    def __init__(self, in_channels, out_channels, dropout=0.0):
        super().__init__()
        layers = [
            nn.ConvTranspose2d(in_channels, out_channels, kernel_size=4, stride=2, padding=1,
bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        ]
        if dropout > 0.0:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)

    def forward(self, x, skip_input):
        x = self.model(x)
        x = torch.cat((x, skip_input), 1)
        return x

class GeneratorUNet(nn.Module):
    def __init__(self, in_channels=1, out_channels=1):
        super().__init__()
        self.down1 = UNetDown(in_channels, 64, normalize=False)
        self.down2 = UNetDown(64, 128)
        self.down3 = UNetDown(128, 256)
        self.down4 = UNetDown(256, 512, dropout=0.5)
        self.down5 = UNetDown(512, 512, dropout=0.5)

        self.up1 = UNetUp(512, 512, dropout=0.5)
        self.up2 = UNetUp(1024, 256)
        self.up3 = UNetUp(512, 128)
        self.up4 = UNetUp(256, 64)

        self.final = nn.Sequential(
            nn.ConvTranspose2d(128, out_channels, kernel_size=4, stride=2, padding=1),
            nn.Tanh()
        )

    def forward(self, x):
        d1 = self.down1(x)
        d2 = self.down2(d1)

```

```

d3 = self.down3(d2)
d4 = self.down4(d3)
d5 = self.down5(d4)

u1 = self.up1(d5, d4)
u2 = self.up2(u1, d3)
u3 = self.up3(u2, d2)
u4 = self.up4(u3, d1)
return self.final(u4)

class Discriminator(nn.Module):
    def __init__(self, in_channels=1):
        super().__init__()
        def discriminator_block(in_filters, out_filters, normalization=True):
            layers = [nn.Conv2d(in_filters, out_filters, 4, stride=2, padding=1)]
            if normalization:
                layers.append(nn.BatchNorm2d(out_filters))
            layers.append(nn.LeakyReLU(0.2, inplace=True))
            return layers

        self.model = nn.Sequential(
            *discriminator_block(in_channels * 2, 64, normalization=False),
            *discriminator_block(64, 128),
            *discriminator_block(128, 256),
            *discriminator_block(256, 512),
            nn.Conv2d(512, 1, 4, padding=1, bias=False)
        )

    def forward(self, img_A, img_B):
        img_input = torch.cat((img_A, img_B), 1)
        return self.model(img_input)

generator = GeneratorUNet().to(device)
discriminator = Discriminator().to(device)
OUTPUT_BASELINE_DIR = "baseline_cGAN_results"
os.makedirs(OUTPUT_BASELINE_DIR, exist_ok=True)

optimizer_G = optim.Adam(generator.parameters(), lr=0.0002, betas=(0.5, 0.999))
optimizer_D = optim.Adam(discriminator.parameters(), lr=0.0002, betas=(0.5, 0.999))

criterion_GAN = nn.BCEWithLogitsLoss()
criterion_pixelwise = nn.L1Loss()
lambda_pixel = 100

```

```

epochs = 200

def show_current_results(test_A, test_B, fake_B, epoch, d_loss, g_loss):
    img_A = test_A[0, 0].cpu().detach().numpy()
    img_B = test_B[0, 0].cpu().detach().numpy()
    img_fake = fake_B[0, 0].cpu().detach().numpy()

    clear_output(wait=True)
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    fig.suptitle(f"Model Evaluation - Epoch {epoch+1} | D Loss: {d_loss:.4f} | G Loss: {g_loss:.4f}", fontsize=16)

    axes[0].imshow(img_A, cmap='magma', origin='lower')
    axes[0].set_title("Input (Degraded)")
    axes[0].axis('off')

    axes[1].imshow(img_B, cmap='magma', origin='lower')
    axes[1].set_title("Ground Truth")
    axes[1].axis('off')

    axes[2].imshow(img_fake, cmap='magma', origin='lower')
    axes[2].set_title("Generated Output")
    axes[2].axis('off')

    plt.tight_layout()
    plt.show()

print("開始訓練 cGAN 模型...")

for epoch in tqdm(range(epochs), desc="Training Progress"):

    generator.train()
    discriminator.train()

    for i, (real_A, real_B) in enumerate(train_dataloader):
        real_A = real_A.to(device)
        real_B = real_B.to(device)

        optimizer_G.zero_grad()
        fake_B = generator(real_A)

        pred_fake = discriminator(real_A, fake_B)
        valid = torch.ones_like(pred_fake, device=device, requires_grad=False)

```

```

loss_GAN = criterion_GAN(pred_fake, valid)

loss_pixel = criterion_pixelwise(fake_B, real_B)
loss_G = loss_GAN + lambda_pixel * loss_pixel

loss_G.backward()
optimizer_G.step()

optimizer_D.zero_grad()

pred_real = discriminator(real_A, real_B)
valid_real = torch.ones_like(pred_real, device=device, requires_grad=False)
loss_real = criterion_GAN(pred_real, valid_real)

pred_fake_detach = discriminator(real_A, fake_B.detach())
fake_label = torch.zeros_like(pred_fake_detach, device=device, requires_grad=False)
loss_fake = criterion_GAN(pred_fake_detach, fake_label)

loss_D = 0.5 * (loss_real + loss_fake)
loss_D.backward()
optimizer_D.step()

generator.eval()
with torch.no_grad():

    test_A, test_B = next(iter(test_dataloader))
    test_A = test_A.to(device)
    test_B = test_B.to(device)
    test_fake_B = generator(test_A)

    if (epoch + 1) % 5 == 0 or epoch == 0:
        show_current_results(test_A, test_B, test_fake_B, epoch, loss_D.item(),
loss_G.item())

    if (epoch + 1) % 40 == 0:
        save_path_npz = os.path.join(OUTPUT_BASELINE_DIR,
f"baseline_ep{epoch+1}_test_data.npz")
        np.savez_compressed(save_path_npz,
                           inputs=test_A.cpu().numpy(),
                           targets=test_B.cpu().numpy(),
                           generated=test_fake_B.cpu().numpy())

        save_path_pth = os.path.join(OUTPUT_BASELINE_DIR,
f"baseline_ep{epoch+1}_weights.pth")

```

```

torch.save(generator.state_dict(), save_path_pth)

print(f"Epoch {epoch+1} 測試數據與模型權重已儲存。")

print("\n 模型訓練完成。")

```

3.PI-cGAN V1:

```

import os
import glob
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader, random_split
import matplotlib.pyplot as plt
from IPython.display import clear_output
from tqdm.notebook import tqdm

# =====
# 1. 環境設定與資料載入
# =====

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"目前使用的運算裝置: {device}")

class DeepSpaceDataset(Dataset):
    def __init__(self, npz_folder):
        self.inputs = []
        self.targets = []
        file_paths = glob.glob(os.path.join(npz_folder, "*.npz"))

        print(f"載入 {len(file_paths)} 個 .npz 資料檔...")
        for f in file_paths:
            data = np.load(f)
            self.inputs.append(data['inputs'])
            self.targets.append(data['targets'])

        self.inputs = np.concatenate(self.inputs, axis=0)
        self.targets = np.concatenate(self.targets, axis=0)
        self.inputs = np.expand_dims(self.inputs, axis=1)
        self.targets = np.expand_dims(self.targets, axis=1)
        print(f"資料載入完成，總切片數: {self.inputs.shape[0]}")

```

```

def __len__(self):
    return self.inputs.shape[0]

def __getitem__(self, idx):
    x = torch.tensor(self.inputs[idx], dtype=torch.float32)
    y = torch.tensor(self.targets[idx], dtype=torch.float32)
    x = torch.clamp((x - 0.5) * 2.0, -1.0, 1.0)
    y = torch.clamp((y - 0.5) * 2.0, -1.0, 1.0)
    return x, y

DATA_DIR = "processed_dataset"
full_dataset = DeepSpaceDataset(DATA_DIR)

dataset_size = len(full_dataset)
test_size = int(0.2 * dataset_size)
train_size = dataset_size - test_size

train_dataset, test_dataset = random_split(full_dataset, [train_size, test_size],
generator=torch.Generator().manual_seed(42))

train_dataloader = DataLoader(train_dataset, batch_size=16, shuffle=True, drop_last=True)
test_dataloader = DataLoader(test_dataset, batch_size=16, shuffle=False, drop_last=False)

print(f"資料集切分: 訓練集 {train_size} 筆, 測試集 {test_size} 筆")

# =====
# 2. 神經網路架構
# =====
class UNetDown(nn.Module):
    def __init__(self, in_channels, out_channels, normalize=True, dropout=0.0):
        super().__init__()
        layers = [nn.Conv2d(in_channels, out_channels, kernel_size=4, stride=2, padding=1,
bias=False)]
        if normalize:
            layers.append(nn.BatchNorm2d(out_channels))
        layers.append(nn.LeakyReLU(0.2))
        if dropout > 0.0:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)

    def forward(self, x):
        return self.model(x)

```



```

class UNetUp(nn.Module):
    def __init__(self, in_channels, out_channels, dropout=0.0):
        super().__init__()
        layers = [
            nn.ConvTranspose2d(in_channels, out_channels, kernel_size=4, stride=2, padding=1,
bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        ]
        if dropout > 0.0:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)

    def forward(self, x, skip_input):
        x = self.model(x)
        x = torch.cat((x, skip_input), 1)
        return x

class GeneratorUNet(nn.Module):
    def __init__(self, in_channels=1, out_channels=1):
        super().__init__()
        self.down1 = UNetDown(in_channels, 64, normalize=False)
        self.down2 = UNetDown(64, 128)
        self.down3 = UNetDown(128, 256)
        self.down4 = UNetDown(256, 512, dropout=0.5)
        self.down5 = UNetDown(512, 512, dropout=0.5)

        self.up1 = UNetUp(512, 512, dropout=0.5)
        self.up2 = UNetUp(1024, 256)
        self.up3 = UNetUp(512, 128)
        self.up4 = UNetUp(256, 64)

        self.final = nn.Sequential(
            nn.ConvTranspose2d(128, out_channels, kernel_size=4, stride=2, padding=1),
            nn.Tanh()
        )

    def forward(self, x):
        d1 = self.down1(x)
        d2 = self.down2(d1)
        d3 = self.down3(d2)
        d4 = self.down4(d3)
        d5 = self.down5(d4)

```

```

u1 = self.up1(d5, d4)
u2 = self.up2(u1, d3)
u3 = self.up3(u2, d2)
u4 = self.up4(u3, d1)
return self.final(u4)

```

```

class Discriminator(nn.Module):
    def __init__(self, in_channels=1):
        super().__init__()
        def discriminator_block(in_filters, out_filters, normalization=True):
            layers = [nn.Conv2d(in_filters, out_filters, 4, stride=2, padding=1)]
            if normalization:
                layers.append(nn.BatchNorm2d(out_filters))
            layers.append(nn.LeakyReLU(0.2, inplace=True))
            return layers

        self.model = nn.Sequential(
            *discriminator_block(in_channels * 2, 64, normalization=False),
            *discriminator_block(64, 128),
            *discriminator_block(128, 256),
            *discriminator_block(256, 512),
            nn.Conv2d(512, 1, 4, padding=1, bias=False)
        )

    def forward(self, img_A, img_B):
        img_input = torch.cat((img_A, img_B), 1)
        return self.model(img_input)

```

```
# =====
```

```
# 3. 模型與物理約束損失函數定義
```

```
# =====
```

```

generator = GeneratorUNet().to(device)
discriminator = Discriminator().to(device)

```

```

class PhysicsInformedLoss(nn.Module):
    def __init__(self, lambda_pixel=100, lambda_fft=10, lambda_flux=5):
        super().__init__()
        self.lambda_pixel = lambda_pixel
        self.lambda_fft = lambda_fft
        self.lambda_flux = lambda_flux
        self.criterion_l1 = nn.L1Loss()

    def forward(self, generated, target):

```

```

# 空間域約束 (L1)
loss_pixel = self.criterion_l1(generated, target)

# 頻率域約束
fft_gen = torch.fft.fft2(generated, norm="ortho")
fft_target = torch.fft.fft2(target, norm="ortho")
loss_fft_real = self.criterion_l1(torch.real(fft_gen), torch.real(fft_target))
loss_fft_imag = self.criterion_l1(torch.imag(fft_gen), torch.imag(fft_target))
loss_fft = loss_fft_real + loss_fft_imag

# 物理域約束
flux_gen = torch.mean(generated, dim=(2, 3))
flux_target = torch.mean(target, dim=(2, 3))
loss_flux = self.criterion_l1(flux_gen, flux_target)

total_pi_loss = (self.lambda_pixel * loss_pixel) + \
    (self.lambda_fft * loss_fft) + \
    (self.lambda_flux * loss_flux)

return total_pi_loss, loss_pixel, loss_fft, loss_flux

pi_criterion = PhysicsInformedLoss(lambda_pixel=100, lambda_fft=10,
lambda_flux=5).to(device)
criterion_GAN = nn.BCEWithLogitsLoss()

OUTPUT_PICGAN_DIR = "PI_cGAN_results"
os.makedirs(OUTPUT_PICGAN_DIR, exist_ok=True)

optimizer_G = optim.Adam(generator.parameters(), lr=0.0002, betas=(0.5, 0.999))
optimizer_D = optim.Adam(discriminator.parameters(), lr=0.0002, betas=(0.5, 0.999))
epochs = 200

# =====
# 4. 訓練監控函數與訓練迴圈
# =====
def show_current_results(test_A, test_B, fake_B, epoch, d_loss, g_loss, fft_loss, flux_loss):
    img_A = test_A[0, 0].cpu().detach().numpy()
    img_B = test_B[0, 0].cpu().detach().numpy()
    img_fake = fake_B[0, 0].cpu().detach().numpy()

    clear_output(wait=True)
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))

```

```
fig.suptitle(f"PI-cGAN Evaluation - Epoch {epoch+1} | D: {d_loss:.4f} | G:
{g_loss:.4f}\nFFT Loss: {fft_loss:.4f} | Flux Loss: {flux_loss:.4f}", fontsize=14)
```

```
axes[0].imshow(img_A, cmap='magma', origin='lower', vmin=-1, vmax=1)
axes[0].set_title("Input (Degraded)")
axes[0].axis('off')
```

```
axes[1].imshow(img_B, cmap='magma', origin='lower', vmin=-1, vmax=1)
axes[1].set_title("Ground Truth")
axes[1].axis('off')
```

```
axes[2].imshow(img_fake, cmap='magma', origin='lower', vmin=-1, vmax=1)
axes[2].set_title("PI-cGAN Generated")
axes[2].axis('off')
```

```
plt.tight_layout()
plt.show()
```

```
print("開始訓練 PI-cGAN 模型...")
```

```
for epoch in tqdm(range(epochs), desc="PI-cGAN Training Progress"):
```

```
    generator.train()
    discriminator.train()
```

```
    epoch_loss_fft = 0.0
    epoch_loss_flux = 0.0
```

```
    for i, (real_A, real_B) in enumerate(train_dataloader):
        real_A = real_A.to(device)
        real_B = real_B.to(device)
```

```
        # 訓練生成器
        optimizer_G.zero_grad()
        fake_B = generator(real_A)
```

```
        pred_fake = discriminator(real_A, fake_B)
        valid = torch.ones_like(pred_fake, device=device, requires_grad=False)
        loss_GAN = criterion_GAN(pred_fake, valid)
```

```
        total_pi_loss, loss_pixel, loss_fft, loss_flux = pi_criterion(fake_B, real_B)
```

```
        loss_G = loss_GAN + total_pi_loss
```

```

loss_G.backward()
optimizer_G.step()

epoch_loss_fft += loss_fft.item()
epoch_loss_flux += loss_flux.item()

# 訓練判別器
optimizer_D.zero_grad()

pred_real = discriminator(real_A, real_B)
valid_real = torch.ones_like(pred_real, device=device, requires_grad=False)
loss_real = criterion_GAN(pred_real, valid_real)

pred_fake_detach = discriminator(real_A, fake_B.detach())
fake_label = torch.zeros_like(pred_fake_detach, device=device, requires_grad=False)
loss_fake = criterion_GAN(pred_fake_detach, fake_label)

loss_D = 0.5 * (loss_real + loss_fake)
loss_D.backward()
optimizer_D.step()

# 測試與驗證
generator.eval()
with torch.no_grad():
    test_A, test_B = next(iter(test_dataloader))
    test_A = test_A.to(device)
    test_B = test_B.to(device)
    test_fake_B = generator(test_A)

avg_fft_loss = epoch_loss_fft / len(train_dataloader)
avg_flux_loss = epoch_loss_flux / len(train_dataloader)

if (epoch + 1) % 5 == 0 or epoch == 0:
    show_current_results(test_A, test_B, test_fake_B, epoch, loss_D.item(),
loss_G.item(), avg_fft_loss, avg_flux_loss)

if (epoch + 1) % 40 == 0:
    save_path_npz = os.path.join(OUTPUT_PICGAN_DIR,
f"picgan_ep{epoch+1}_test_data.npz")
    np.savez_compressed(save_path_npz,
                        inputs=test_A.cpu().numpy(),
                        targets=test_B.cpu().numpy(),
                        generated=test_fake_B.cpu().numpy())

```

```

        save_path_pth = os.path.join(OUTPUT_PICGAN_DIR,
f"picgan_ep{epoch+1}_weights.pth")
        torch.save(generator.state_dict(), save_path_pth)

    print(f"Epoch {epoch+1} PI-cGAN 測試數據與模型權重已儲存。")

print("\nPI-cGAN 模型訓練完成。")

```

4.PI-cGAN V2:

```

import os
import glob
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split
import matplotlib.pyplot as plt
from IPython.display import clear_output
from tqdm.notebook import tqdm

# =====
# 1. 環境設定與資料載入
# =====
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"目前使用的運算裝置: {device}")

class DeepSpaceDataset(Dataset):
    def __init__(self, npz_folder):
        self.inputs = []
        self.targets = []
        file_paths = glob.glob(os.path.join(npz_folder, "*.npz"))

        print(f"正在載入 {len(file_paths)} 個 .npz 資料檔...")
        for f in file_paths:
            data = np.load(f)
            self.inputs.append(data['inputs'])
            self.targets.append(data['targets'])

        self.inputs = np.concatenate(self.inputs, axis=0)
        self.targets = np.concatenate(self.targets, axis=0)
        self.inputs = np.expand_dims(self.inputs, axis=1)
        self.targets = np.expand_dims(self.targets, axis=1)

```

```

print(f"資料載入完成，總切片數: {self.inputs.shape[0]}")

def __len__(self):
    return self.inputs.shape[0]

def __getitem__(self, idx):
    x = torch.tensor(self.inputs[idx], dtype=torch.float32)
    y = torch.tensor(self.targets[idx], dtype=torch.float32)
    x = torch.clamp((x - 0.5) * 2.0, -1.0, 1.0)
    y = torch.clamp((y - 0.5) * 2.0, -1.0, 1.0)
    return x, y

DATA_DIR = "processed_dataset"
full_dataset = DeepSpaceDataset(DATA_DIR)

dataset_size = len(full_dataset)
test_size = int(0.2 * dataset_size)
train_size = dataset_size - test_size

train_dataset, test_dataset = random_split(
    full_dataset, [train_size, test_size],
    generator=torch.Generator().manual_seed(42)
)

train_dataloader = DataLoader(train_dataset, batch_size=16, shuffle=True, drop_last=True)
test_dataloader = DataLoader(test_dataset, batch_size=16, shuffle=False, drop_last=False)

print(f"資料集切分: 訓練集 {train_size} 筆, 測試集 {test_size} 筆")

# =====
# 2. 神經網路架構
# =====
class UNetDown(nn.Module):
    def __init__(self, in_channels, out_channels, normalize=True, dropout=0.0):
        super().__init__()
        layers = [nn.Conv2d(in_channels, out_channels, kernel_size=4, stride=2, padding=1,
bias=False)]
        if normalize:
            layers.append(nn.BatchNorm2d(out_channels))
        layers.append(nn.LeakyReLU(0.2))
        if dropout > 0.0:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)

```

```

def forward(self, x):
    return self.model(x)

class UNetUp(nn.Module):
    def __init__(self, in_channels, out_channels, dropout=0.0):
        super().__init__()
        layers = [
            nn.ConvTranspose2d(in_channels, out_channels, kernel_size=4, stride=2, padding=1,
bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        ]
        if dropout > 0.0:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)

    def forward(self, x, skip_input):
        x = self.model(x)
        x = torch.cat((x, skip_input), 1)
        return x

class GeneratorUNet(nn.Module):
    def __init__(self, in_channels=1, out_channels=1):
        super().__init__()
        self.down1 = UNetDown(in_channels, 64, normalize=False)
        self.down2 = UNetDown(64, 128)
        self.down3 = UNetDown(128, 256)
        self.down4 = UNetDown(256, 512, dropout=0.5)
        self.down5 = UNetDown(512, 512, dropout=0.5)

        self.up1 = UNetUp(512, 512, dropout=0.5)
        self.up2 = UNetUp(1024, 256)
        self.up3 = UNetUp(512, 128)
        self.up4 = UNetUp(256, 64)

        self.final = nn.Sequential(
            nn.ConvTranspose2d(128, out_channels, kernel_size=4, stride=2, padding=1),
            nn.Tanh()
        )

    def forward(self, x):
        d1 = self.down1(x)
        d2 = self.down2(d1)

```



```

d3 = self.down3(d2)
d4 = self.down4(d3)
d5 = self.down5(d4)

u1 = self.up1(d5, d4)
u2 = self.up2(u1, d3)
u3 = self.up3(u2, d2)
u4 = self.up4(u3, d1)
return self.final(u4)

class Discriminator(nn.Module):
    def __init__(self, in_channels=1):
        super().__init__()
        def discriminator_block(in_filters, out_filters, normalization=True):
            layers = [nn.Conv2d(in_filters, out_filters, 4, stride=2, padding=1)]
            if normalization:
                layers.append(nn.BatchNorm2d(out_filters))
            layers.append(nn.LeakyReLU(0.2, inplace=True))
            return layers

        self.model = nn.Sequential(
            *discriminator_block(in_channels * 2, 64, normalization=False),
            *discriminator_block(64, 128),
            *discriminator_block(128, 256),
            *discriminator_block(256, 512),
            nn.Conv2d(512, 1, 4, padding=1, bias=False)
        )

    def forward(self, img_A, img_B):
        img_input = torch.cat((img_A, img_B), 1)
        return self.model(img_input)

# =====
# 3. 模型與物理約束損失函數定義
# =====
generator = GeneratorUNet().to(device)
discriminator = Discriminator().to(device)

class PhysicsInformedLossV2(nn.Module):
    def __init__(self, lambda_pixel=100, lambda_fft=0.1, lambda_flux=0.01):
        super().__init__()
        self.lambda_pixel = lambda_pixel
        self.lambda_fft = lambda_fft

```

```

self.lambda_flux = lambda_flux
self.criterion_l1 = nn.L1Loss()

def forward(self, generated, target):
    # 空間域約束 (L1)
    loss_pixel = self.criterion_l1(generated, target)

    # 頻率域約束
    fft_gen = torch.fft.fft2(generated, norm="ortho")
    fft_target = torch.fft.fft2(target, norm="ortho")
    loss_fft_real = self.criterion_l1(torch.real(fft_gen), torch.real(fft_target))
    loss_fft_imag = self.criterion_l1(torch.imag(fft_gen), torch.imag(fft_target))
    loss_fft = loss_fft_real + loss_fft_imag

    # 物理域約束 (GT 靜態遮罩)
    star_mask = (target > 0.0).float().detach()
    flux_gen = torch.sum(generated * star_mask, dim=(2, 3))
    flux_target = torch.sum(target * star_mask, dim=(2, 3))
    loss_flux = self.criterion_l1(flux_gen, flux_target)

    total_pi_loss = (self.lambda_pixel * loss_pixel) + \
                    (self.lambda_fft * loss_fft) + \
                    (self.lambda_flux * loss_flux)

    return total_pi_loss, loss_pixel, loss_fft, loss_flux

pi_criterion = PhysicsInformedLossV2(lambda_pixel=100, lambda_fft=0.1,
lambda_flux=0.01).to(device)
criterion_pixelwise = nn.L1Loss().to(device)
criterion_GAN = nn.BCEWithLogitsLoss().to(device)

OUTPUT_PICGAN_DIR = "PI_cGAN_V2_Optimized_results"
os.makedirs(OUTPUT_PICGAN_DIR, exist_ok=True)
print(f"已建立專屬存檔資料夾：{OUTPUT_PICGAN_DIR}")

optimizer_G = optim.Adam(generator.parameters(), lr=0.0002, betas=(0.5, 0.999))
optimizer_D = optim.Adam(discriminator.parameters(), lr=0.0002, betas=(0.5, 0.999))
epochs = 200

# =====
# 4. 訓練監控函數與課程學習訓練迴圈
# =====

```

```

def show_current_results(test_A, test_B, fake_B, epoch, d_loss, g_loss, fft_loss, flux_loss,
stage_name):
    img_A = test_A[0, 0].cpu().detach().numpy()
    img_B = test_B[0, 0].cpu().detach().numpy()
    img_fake = fake_B[0, 0].cpu().detach().numpy()

    clear_output(wait=True)
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))

    fig.suptitle(f"[{stage_name}] Epoch {epoch+1} | D: {d_loss:.4f} | G: {g_loss:.4f}\nFFT
Loss: {fft_loss:.4f} | Masked Flux Loss: {flux_loss:.4f}", fontsize=14)

    axes[0].imshow(img_A, cmap='magma', origin='lower', vmin=-1, vmax=1)
    axes[0].set_title("Input (Degraded)")
    axes[0].axis('off')

    axes[1].imshow(img_B, cmap='magma', origin='lower', vmin=-1, vmax=1)
    axes[1].set_title("Ground Truth")
    axes[1].axis('off')

    axes[2].imshow(img_fake, cmap='magma', origin='lower', vmin=-1, vmax=1)
    axes[2].set_title("PI-cGAN V2 Generated")
    axes[2].axis('off')

    plt.tight_layout()
    plt.show()

print("開始訓練 PI-cGAN V2 模型...")

warmup_epochs = 100

for epoch in tqdm(range(epochs), desc="PI-cGAN V2 Training Progress"):

    generator.train()
    discriminator.train()

    epoch_loss_fft = 0.0
    epoch_loss_flux = 0.0

    for i, (real_A, real_B) in enumerate(train_dataloader):
        real_A, real_B = real_A.to(device), real_B.to(device)

        # 訓練生成器

```

```

optimizer_G.zero_grad()
fake_B = generator(real_A)

pred_fake = discriminator(real_A, fake_B)
valid = torch.ones_like(pred_fake, device=device, requires_grad=False)
loss_GAN = criterion_GAN(pred_fake, valid)

if epoch < warmup_epochs:
    loss_pixel = criterion_pixelwise(fake_B, real_B)
    loss_G = loss_GAN + (100 * loss_pixel)
    loss_fft_val, loss_flux_val = 0.0, 0.0
else:
    total_pi_loss, loss_pixel, loss_fft, loss_flux = pi_criterion(fake_B, real_B)
    loss_G = loss_GAN + total_pi_loss
    loss_fft_val = loss_fft.item()
    loss_flux_val = loss_flux.item()

loss_G.backward()
optimizer_G.step()

epoch_loss_fft += loss_fft_val
epoch_loss_flux += loss_flux_val

# 訓練判別器
optimizer_D.zero_grad()
pred_real = discriminator(real_A, real_B)
loss_real = criterion_GAN(pred_real, torch.ones_like(pred_real))
pred_fake_detach = discriminator(real_A, fake_B.detach())
loss_fake = criterion_GAN(pred_fake_detach, torch.zeros_like(pred_fake_detach))

loss_D = 0.5 * (loss_real + loss_fake)
loss_D.backward()
optimizer_D.step()

# 測試與驗證
generator.eval()
with torch.no_grad():
    test_A, test_B = next(iter(test_dataloader))
    test_A, test_B = test_A.to(device), test_B.to(device)
    test_fake_B = generator(test_A)

avg_fft_loss = epoch_loss_fft / len(train_dataloader)
avg_flux_loss = epoch_loss_flux / len(train_dataloader)

```

```

    if (epoch + 1) % 5 == 0 or epoch == 0:
        stage_name = "Stage 1: Warmup" if epoch < warmup_epochs else "Stage 2: Physics-
Informed"
        show_current_results(test_A, test_B, test_fake_B, epoch, loss_D.item(),
loss_G.item(), avg_fft_loss, avg_flux_loss, stage_name)

    if (epoch + 1) % 40 == 0:
        save_path_npz = os.path.join(OUTPUT_PICGAN_DIR,
f"picgan_ep{epoch+1}_test_data.npz")
        np.savez_compressed(save_path_npz, inputs=test_A.cpu().numpy(),
targets=test_B.cpu().numpy(), generated=test_fake_B.cpu().numpy())

        save_path_pth = os.path.join(OUTPUT_PICGAN_DIR,
f"picgan_ep{epoch+1}_weights.pth")
        torch.save(generator.state_dict(), save_path_pth)
        print(f"Epoch {epoch+1} 測試數據已儲存至 {OUTPUT_PICGAN_DIR} 。")

print("\nPI-cGAN V2 訓練完成。")

```

5.PI-cGAN V3:

```

import os
import glob
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split
import matplotlib.pyplot as plt
from IPython.display import clear_output
from tqdm.notebook import tqdm

# =====
# 1. 環境設定與資料載入
# =====
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"目前使用的運算裝置: {device}")

class DeepSpaceDataset(Dataset):
    def __init__(self, npz_folder):
        self.inputs = []
        self.targets = []
        file_paths = glob.glob(os.path.join(npz_folder, "*.npz"))

```

```

print(f"正在載入 {len(file_paths)} 個 .npz 資料檔...")
for f in file_paths:
    data = np.load(f)
    self.inputs.append(data['inputs'])
    self.targets.append(data['targets'])

self.inputs = np.concatenate(self.inputs, axis=0)
self.targets = np.concatenate(self.targets, axis=0)
self.inputs = np.expand_dims(self.inputs, axis=1)
self.targets = np.expand_dims(self.targets, axis=1)
print(f"資料載入完成，總切片數: {self.inputs.shape[0]}")

def __len__(self):
    return self.inputs.shape[0]

def __getitem__(self, idx):
    x = torch.tensor(self.inputs[idx], dtype=torch.float32)
    y = torch.tensor(self.targets[idx], dtype=torch.float32)

    # 資料標準化映射至 [-1.0, 1.0]
    x = (x - 0.5) / 0.55
    y = (y - 0.5) / 0.55

    x = torch.clamp(x, -1.0, 1.0)
    y = torch.clamp(y, -1.0, 1.0)
    return x, y

DATA_DIR = "processed_dataset"
full_dataset = DeepSpaceDataset(DATA_DIR)

dataset_size = len(full_dataset)
test_size = int(0.2 * dataset_size)
train_size = dataset_size - test_size

train_dataset, test_dataset = random_split(full_dataset, [train_size, test_size],
generator=torch.Generator().manual_seed(42))
train_dataloader = DataLoader(train_dataset, batch_size=16, shuffle=True, drop_last=True)
test_dataloader = DataLoader(test_dataset, batch_size=16, shuffle=False, drop_last=False)

print(f"資料集切分: 訓練集 {train_size} 筆, 測試集 {test_size} 筆")

# =====
# 2. 神經網路架構

```

```

# =====
class UNetDown(nn.Module):
    def __init__(self, in_channels, out_channels, normalize=True, dropout=0.0):
        super().__init__()
        layers = [nn.Conv2d(in_channels, out_channels, kernel_size=4, stride=2, padding=1,
bias=False)]
        if normalize:
            layers.append(nn.BatchNorm2d(out_channels))
        layers.append(nn.LeakyReLU(0.2))
        if dropout > 0.0:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)

    def forward(self, x):
        return self.model(x)

class UNetUp(nn.Module):
    def __init__(self, in_channels, out_channels, dropout=0.0):
        super().__init__()
        layers = [
            nn.ConvTranspose2d(in_channels, out_channels, kernel_size=4, stride=2, padding=1,
bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        ]
        if dropout > 0.0:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)

    def forward(self, x, skip_input):
        x = self.model(x)
        x = torch.cat((x, skip_input), 1)
        return x

class GeneratorUNet(nn.Module):
    def __init__(self, in_channels=1, out_channels=1):
        super().__init__()
        self.down1 = UNetDown(in_channels, 64, normalize=False)
        self.down2 = UNetDown(64, 128)
        self.down3 = UNetDown(128, 256)
        self.down4 = UNetDown(256, 512, dropout=0.5)
        self.down5 = UNetDown(512, 512, dropout=0.5)
        self.up1 = UNetUp(512, 512, dropout=0.5)
        self.up2 = UNetUp(1024, 256)

```

```

self.up3 = UNetUp(512, 128)
self.up4 = UNetUp(256, 64)
self.final = nn.Sequential(nn.ConvTranspose2d(128, out_channels, kernel_size=4,
stride=2, padding=1), nn.Tanh())

```

```

def forward(self, x):
    d1 = self.down1(x)
    d2 = self.down2(d1)
    d3 = self.down3(d2)
    d4 = self.down4(d3)
    d5 = self.down5(d4)
    u1 = self.up1(d5, d4)
    u2 = self.up2(u1, d3)
    u3 = self.up3(u2, d2)
    u4 = self.up4(u3, d1)
    return self.final(u4)

```

```

class Discriminator(nn.Module):
    def __init__(self, in_channels=1):
        super().__init__()
        def discriminator_block(in_filters, out_filters, normalization=True):
            layers = [nn.Conv2d(in_filters, out_filters, 4, stride=2, padding=1)]
            if normalization: layers.append(nn.BatchNorm2d(out_filters))
            layers.append(nn.LeakyReLU(0.2, inplace=True))
            return layers
        self.model = nn.Sequential(
            *discriminator_block(in_channels * 2, 64, normalization=False),
            *discriminator_block(64, 128),
            *discriminator_block(128, 256),
            *discriminator_block(256, 512),
            nn.Conv2d(512, 1, 4, padding=1, bias=False)
        )

```

```

def forward(self, img_A, img_B):
    img_input = torch.cat((img_A, img_B), 1)
    return self.model(img_input)

```

```

# =====
# 3. 模型與物理約束損失函數定義
# =====
generator = GeneratorUNet().to(device)
discriminator = Discriminator().to(device)

```



```

class PhysicsInformedLossV3(nn.Module):
    def __init__(self, lambda_pixel=100, lambda_fft=0.1, lambda_flux=0.01, asinh_a=0.1):
        super().__init__()
        self.lambda_pixel = lambda_pixel
        self.lambda_fft = lambda_fft
        self.lambda_flux = lambda_flux
        self.criterion_l1 = nn.L1Loss()

        self.a = asinh_a
        self.arcsinh_1_over_a = np.arcsinh(1.0 / self.a)

    def inverse_transform_to_linear(self, tensor_img):
        y_original = (tensor_img * 0.55) + 0.5
        y_clamped = torch.clamp(y_original, 0.0, 1.0)
        linear_space_img = self.a * torch.sinh(y_clamped * self.arcsinh_1_over_a)
        return linear_space_img

    def forward(self, generated, target):
        gen_linear = self.inverse_transform_to_linear(generated)
        target_linear = self.inverse_transform_to_linear(target)

        # 空間域約束 (線性空間計算)
        loss_pixel = self.criterion_l1(gen_linear, target_linear)

        # 頻率域約束
        fft_gen = torch.fft.fft2(generated, norm="ortho")
        fft_target = torch.fft.fft2(target, norm="ortho")
        loss_fft_real = self.criterion_l1(torch.real(fft_gen), torch.real(fft_target))
        loss_fft_imag = self.criterion_l1(torch.imag(fft_gen), torch.imag(fft_target))
        loss_fft = loss_fft_real + loss_fft_imag

        # 物理光通量約束
        star_mask = (target > 0.0).float().detach()
        flux_gen = torch.sum(gen_linear * star_mask, dim=(2, 3))
        flux_target = torch.sum(target_linear * star_mask, dim=(2, 3))
        loss_flux = self.criterion_l1(flux_gen, flux_target)

        total_pi_loss = (self.lambda_pixel * loss_pixel) + \
            (self.lambda_fft * loss_fft) + \
            (self.lambda_flux * loss_flux)

        return total_pi_loss, loss_pixel, loss_fft, loss_flux

```

```

pi_criterion = PhysicsInformedLossV3(lambda_pixel=100, lambda_fft=0.1,
lambda_flux=0.01).to(device)
criterion_pixelwise = nn.L1Loss().to(device)
criterion_GAN = nn.BCEWithLogitsLoss().to(device)

OUTPUT_PICGAN_DIR = "PI_cGAN_V3_Optimized_results"
os.makedirs(OUTPUT_PICGAN_DIR, exist_ok=True)
print(f"已建立存檔資料夾：{OUTPUT_PICGAN_DIR}")

optimizer_G = optim.Adam(generator.parameters(), lr=0.0002, betas=(0.5, 0.999))
optimizer_D = optim.Adam(discriminator.parameters(), lr=0.0002, betas=(0.5, 0.999))
epochs = 200

# =====
# 4. 訓練監控與訓練迴圈
# =====
def show_current_results(test_A, test_B, fake_B, epoch, d_loss, g_loss, fft_loss, flux_loss,
stage_name):
    img_A = test_A[0, 0].cpu().detach().numpy()
    img_B = test_B[0, 0].cpu().detach().numpy()
    img_fake = fake_B[0, 0].cpu().detach().numpy()

    clear_output(wait=True)
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    fig.suptitle(f"[{stage_name}] Epoch {epoch+1} | D: {d_loss:.4f} | G: {g_loss:.4f}\nFFT:
{fft_loss:.4f} | Flux: {flux_loss:.4f}", fontsize=14)

    axes[0].imshow(img_A, cmap='magma', origin='lower', vmin=-1, vmax=1)
    axes[0].set_title("Input (Degraded)")
    axes[0].axis('off')

    axes[1].imshow(img_B, cmap='magma', origin='lower', vmin=-1, vmax=1)
    axes[1].set_title("Ground Truth")
    axes[1].axis('off')

    axes[2].imshow(img_fake, cmap='magma', origin='lower', vmin=-1, vmax=1)
    axes[2].set_title("Generated Output")
    axes[2].axis('off')

    plt.tight_layout()
    plt.show()

print("開始訓練 PI-cGAN 模型...")

```

```

warmup_epochs = 100

for epoch in tqdm(range(epochs), desc="PI-cGAN Training Progress"):
    generator.train()
    discriminator.train()

    epoch_loss_fft = 0.0
    epoch_loss_flux = 0.0

    for i, (real_A, real_B) in enumerate(train_dataloader):
        real_A, real_B = real_A.to(device), real_B.to(device)

        # 訓練生成器
        optimizer_G.zero_grad()
        fake_B = generator(real_A)

        pred_fake = discriminator(real_A, fake_B)
        valid = torch.ones_like(pred_fake, device=device, requires_grad=False)
        loss_GAN = criterion_GAN(pred_fake, valid)

        if epoch < warmup_epochs:
            loss_pixel = criterion_pixelwise(fake_B, real_B)
            loss_G = loss_GAN + (100 * loss_pixel)
            loss_fft_val, loss_flux_val = 0.0, 0.0
        else:
            total_pi_loss, loss_pixel, loss_fft, loss_flux = pi_criterion(fake_B, real_B)
            loss_G = loss_GAN + total_pi_loss
            loss_fft_val = loss_fft.item()
            loss_flux_val = loss_flux.item()

        loss_G.backward()
        optimizer_G.step()

    epoch_loss_fft += loss_fft_val
    epoch_loss_flux += loss_flux_val

    # 訓練判別器
    optimizer_D.zero_grad()
    pred_real = discriminator(real_A, real_B)
    loss_real = criterion_GAN(pred_real, torch.ones_like(pred_real))
    pred_fake_detach = discriminator(real_A, fake_B.detach())
    loss_fake = criterion_GAN(pred_fake_detach, torch.zeros_like(pred_fake_detach))
    loss_D = 0.5 * (loss_real + loss_fake)

```

```

loss_D.backward()
optimizer_D.step()

# 推論與存檔
generator.eval()
with torch.no_grad():
    test_A, test_B = next(iter(test_dataloader))
    test_A, test_B = test_A.to(device), test_B.to(device)
    test_fake_B = generator(test_A)

    avg_fft_loss = epoch_loss_fft / len(train_dataloader)
    avg_flux_loss = epoch_loss_flux / len(train_dataloader)

    if (epoch + 1) % 5 == 0 or epoch == 0:
        stage_name = "Stage 1: Warmup" if epoch < warmup_epochs else "Stage 2: Physics
Active"
        show_current_results(test_A, test_B, test_fake_B, epoch, loss_D.item(),
loss_G.item(), avg_fft_loss, avg_flux_loss, stage_name)

    if (epoch + 1) % 40 == 0:
        save_path_npz = os.path.join(OUTPUT_PICGAN_DIR,
f"picgan_ep{epoch+1}_test_data.npz")
        np.savez_compressed(save_path_npz, inputs=test_A.cpu().numpy(),
targets=test_B.cpu().numpy(), generated=test_fake_B.cpu().numpy())
        save_path_pth = os.path.join(OUTPUT_PICGAN_DIR,
f"picgan_ep{epoch+1}_weights.pth")
        torch.save(generator.state_dict(), save_path_pth)
        print(f"Epoch {epoch+1} 測試數據已存檔。")

print("\n PI-cGAN V3 訓練完成。")

```

6.PI-cGAN V4:

```

import os
import glob
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split
import matplotlib.pyplot as plt
from IPython.display import clear_output
from tqdm.notebook import tqdm
from skimage.metrics import structural_similarity as ssim_metric

```

```

try:
    from pytorch_msssim import ms_ssim
except ImportError:
    raise ImportError("請先安裝 pytorch-msssim: !pip install pytorch-msssim")

# =====
# 1. 環境設定與資料載入
# =====
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"目前使用的運算裝置: {device}")

class DeepSpaceDataset(Dataset):
    def __init__(self, npz_folder):
        self.inputs = []
        self.targets = []
        file_paths = glob.glob(os.path.join(npz_folder, "*.npz"))
        for f in file_paths:
            data = np.load(f)
            self.inputs.append(data['inputs'])
            self.targets.append(data['targets'])
        self.inputs = np.concatenate(self.inputs, axis=0)
        self.targets = np.concatenate(self.targets, axis=0)
        self.inputs = np.expand_dims(self.inputs, axis=1)
        self.targets = np.expand_dims(self.targets, axis=1)

    def __len__(self):
        return self.inputs.shape[0]

    def __getitem__(self, idx):
        x = torch.tensor(self.inputs[idx], dtype=torch.float32)
        y = torch.tensor(self.targets[idx], dtype=torch.float32)
        # 資料標準化映射至 [-1.0, 1.0]
        x = (x - 0.5) / 0.55
        y = (y - 0.5) / 0.55
        x = torch.clamp(x, -1.0, 1.0)
        y = torch.clamp(y, -1.0, 1.0)
        return x, y

DATA_DIR = "processed_dataset"
full_dataset = DeepSpaceDataset(DATA_DIR)

dataset_size = len(full_dataset)
test_size = int(0.2 * dataset_size)

```

```

train_size = dataset_size - test_size

train_dataset, test_dataset = random_split(full_dataset, [train_size, test_size],
generator=torch.Generator().manual_seed(42))
train_dataloader = DataLoader(train_dataset, batch_size=16, shuffle=True, drop_last=True)
test_dataloader = DataLoader(test_dataset, batch_size=16, shuffle=False, drop_last=False)

# =====
# 2. 神經網路架構
# =====
class UNetDown(nn.Module):
    def __init__(self, in_channels, out_channels, normalize=True, dropout=0.0):
        super().__init__()
        layers = [nn.Conv2d(in_channels, out_channels, kernel_size=4, stride=2, padding=1,
bias=False)]
        if normalize:
            layers.append(nn.BatchNorm2d(out_channels))
        layers.append(nn.LeakyReLU(0.2))
        if dropout > 0.0:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)
    def forward(self, x): return self.model(x)

class UNetUp(nn.Module):
    def __init__(self, in_channels, out_channels, dropout=0.0):
        super().__init__()
        layers = [
            nn.ConvTranspose2d(in_channels, out_channels, kernel_size=4, stride=2, padding=1,
bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        ]
        if dropout > 0.0:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)
    def forward(self, x, skip_input):
        x = self.model(x)
        return torch.cat((x, skip_input), 1)

class GeneratorUNet(nn.Module):
    def __init__(self, in_channels=1, out_channels=1):
        super().__init__()
        self.down1 = UNetDown(in_channels, 64, normalize=False)

```

```

self.down2 = UNetDown(64, 128)
self.down3 = UNetDown(128, 256)
self.down4 = UNetDown(256, 512, dropout=0.5)
self.down5 = UNetDown(512, 512, dropout=0.5)
self.up1 = UNetUp(512, 512, dropout=0.5)
self.up2 = UNetUp(1024, 256)
self.up3 = UNetUp(512, 128)
self.up4 = UNetUp(256, 64)
self.final = nn.Sequential(nn.ConvTranspose2d(128, out_channels, kernel_size=4,
stride=2, padding=1), nn.Tanh())
def forward(self, x):
    d1 = self.down1(x)
    d2 = self.down2(d1)
    d3 = self.down3(d2)
    d4 = self.down4(d3)
    d5 = self.down5(d4)
    u1 = self.up1(d5, d4)
    u2 = self.up2(u1, d3)
    u3 = self.up3(u2, d2)
    u4 = self.up4(u3, d1)
    return self.final(u4)

class Discriminator(nn.Module):
    def __init__(self, in_channels=1):
        super().__init__()
        def discriminator_block(in_filters, out_filters, normalization=True):
            layers = [nn.Conv2d(in_filters, out_filters, 4, stride=2, padding=1)]
            if normalization: layers.append(nn.BatchNorm2d(out_filters))
            layers.append(nn.LeakyReLU(0.2, inplace=True))
            return layers
        self.model = nn.Sequential(
            *discriminator_block(in_channels * 2, 64, normalization=False),
            *discriminator_block(64, 128),
            *discriminator_block(128, 256),
            *discriminator_block(256, 512),
            nn.Conv2d(512, 1, 4, padding=1, bias=False)
        )
    def forward(self, img_A, img_B):
        img_input = torch.cat((img_A, img_B), 1)
        return self.model(img_input)

```

```

# =====
# 3. 物理約束損失函數

```

```

# =====
class PhysicsInformedLossV3(nn.Module):
    def __init__(self, lambda_pixel=100, lambda_fft=0.1, lambda_flux=0.01, asinh_a=0.1):
        super().__init__()
        self.lambda_pixel = lambda_pixel
        self.lambda_fft = lambda_fft
        self.lambda_flux = lambda_flux
        self.criterion_l1 = nn.L1Loss()
        self.a = asinh_a
        self.arcsinh_1_over_a = np.arcsinh(1.0 / self.a)

    def inverse_transform_to_linear(self, tensor_img):
        y_original = (tensor_img * 0.55) + 0.5
        y_clamped = torch.clamp(y_original, 0.0, 1.0)
        return self.a * torch.sinh(y_clamped * self.arcsinh_1_over_a)

    def forward(self, generated, target):
        gen_linear = self.inverse_transform_to_linear(generated)
        target_linear = self.inverse_transform_to_linear(target)

        loss_pixel = self.criterion_l1(gen_linear, target_linear)
        fft_gen = torch.fft.fft2(generated, norm="ortho")
        fft_target = torch.fft.fft2(target, norm="ortho")
        loss_fft_real = self.criterion_l1(torch.real(fft_gen), torch.real(fft_target))
        loss_fft_imag = self.criterion_l1(torch.imag(fft_gen), torch.imag(fft_target))
        loss_fft = loss_fft_real + loss_fft_imag

        star_mask = (target > 0.0).float().detach()
        flux_gen = torch.sum(gen_linear * star_mask, dim=(2, 3))
        flux_target = torch.sum(target_linear * star_mask, dim=(2, 3))
        loss_flux = self.criterion_l1(flux_gen, flux_target)

        total_pi_loss = (self.lambda_pixel * loss_pixel) + (self.lambda_fft * loss_fft) +
        (self.lambda_flux * loss_flux)
        return total_pi_loss, loss_pixel, loss_fft, loss_flux

# =====
# 4. 微調環境設定與權重載入
# =====
OUTPUT_V4_DIR = "PI_cGAN_V4_Finetune_results"
os.makedirs(OUTPUT_V4_DIR, exist_ok=True)

generator_v4 = GeneratorUNet().to(device)

```



```

discriminator_v4 = Discriminator().to(device)

BEST_V3_WEIGHTS = "PI_cGAN_V3_Ultimate_results/picgan_ep120_weights.pth"
generator_v4.load_state_dict(torch.load(BEST_V3_WEIGHTS, map_location=device))

LR_V4 = 0.00002
optimizer_G_v4 = optim.Adam(generator_v4.parameters(), lr=LR_V4, betas=(0.5, 0.999))
optimizer_D_v4 = optim.Adam(discriminator_v4.parameters(), lr=LR_V4, betas=(0.5,
0.999))

pi_criterion_v4 = PhysicsInformedLossV3(lambda_pixel=100, lambda_fft=0.1,
lambda_flux=0.01).to(device)
criterion_GAN_v4 = nn.BCEWithLogitsLoss().to(device)

# =====
# 5. 微調訓練迴圈
# =====
epochs_v4 = 50
best_val_ssim = 0.0
patience = 10
no_improve_count = 0

for epoch in range(epochs_v4):
    generator_v4.train()
    discriminator_v4.train()
    epoch_ms_ssim_loss = 0.0

    for i, (real_A, real_B) in enumerate(train_dataloader):
        real_A, real_B = real_A.to(device), real_B.to(device)

        optimizer_G_v4.zero_grad()
        fake_B = generator_v4(real_A)

        pred_fake = discriminator_v4(real_A, fake_B)
        loss_GAN = criterion_GAN_v4(pred_fake, torch.ones_like(pred_fake, device=device))

        total_pi_loss, _, _, _ = pi_criterion_v4(fake_B, real_B)

        gen_01 = (fake_B + 1.0) / 2.0
        real_01 = (real_B + 1.0) / 2.0
        loss_ms_ssim = 1 - ms_ssim(gen_01, real_01, data_range=1.0, size_average=True)

        loss_G = loss_GAN + total_pi_loss + (50 * loss_ms_ssim)

```

```

loss_G.backward()
optimizer_G_v4.step()

epoch_ms_ssim_loss += loss_ms_ssim.item()

optimizer_D_v4.zero_grad()
pred_real = discriminator_v4(real_A, real_B)
loss_real = criterion_GAN_v4(pred_real, torch.ones_like(pred_real))
pred_fake_detach = discriminator_v4(real_A, fake_B.detach())
loss_fake = criterion_GAN_v4(pred_fake_detach, torch.zeros_like(pred_fake_detach))
loss_D = 0.5 * (loss_real + loss_fake)
loss_D.backward()
optimizer_D_v4.step()

generator_v4.eval()
val_ssim_list = []

with torch.no_grad():
    for test_A, test_B in test_dataloader:
        test_A, test_B = test_A.to(device), test_B.to(device)
        test_fake_B = generator_v4(test_A)

        np_fake = test_fake_B.cpu().numpy()
        np_real = test_B.cpu().numpy()
        for b in range(np_fake.shape[0]):
            val_ssim_list.append(ssim_metric(np_real[b, 0], np_fake[b, 0], data_range=2.0))

current_epoch_ssim = np.mean(val_ssim_list)

print(f"Epoch [{epoch+1}/{epochs_v4}] | Validation SSIM: {current_epoch_ssim:.4f}")

if current_epoch_ssim > best_val_ssim:
    best_val_ssim = current_epoch_ssim
    no_improve_count = 0
    best_ptn_path = os.path.join(OUTPUT_V4_DIR, "v4_best_finetuned_weights.pth")
    torch.save(generator_v4.state_dict(), best_ptn_path)

    best_npz_path = os.path.join(OUTPUT_V4_DIR, "v4_best_test_data.npz")
    test_A_sample, test_B_sample = next(iter(test_dataloader))
    test_fake_sample = generator_v4(test_A_sample.to(device))
    np.savez_compressed(best_npz_path,
                        inputs=test_A_sample.cpu().numpy(),
                        targets=test_B_sample.cpu().numpy(),
                        generated=test_fake_sample.detach().cpu().numpy())

```

```

else:
    no_improve_count += 1

if no_improve_count >= patience:
    break

print("PI-cGAN V4 訓練結束。")
7.PI-cGAN V5:
import os
import glob
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split
import matplotlib.pyplot as plt
from IPython.display import clear_output
from tqdm.notebook import tqdm
from skimage.metrics import structural_similarity as ssim_metric

try:
    from pytorch_msssim import ms_ssim
except ImportError:
    raise ImportError("請先安裝 pytorch-msssim: !pip install pytorch-msssim")

# =====
# 1. 環境設定與資料載入
# =====
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"目前使用的運算裝置: {device}")

class DeepSpaceDataset(Dataset):
    def __init__(self, npz_folder):
        self.inputs = []
        self.targets = []
        file_paths = glob.glob(os.path.join(npz_folder, "*.npz"))
        for f in file_paths:
            data = np.load(f)
            self.inputs.append(data['inputs'])
            self.targets.append(data['targets'])
        self.inputs = np.concatenate(self.inputs, axis=0)
        self.targets = np.concatenate(self.targets, axis=0)
        self.inputs = np.expand_dims(self.inputs, axis=1)

```

```

        self.targets = np.expand_dims(self.targets, axis=1)

    def __len__(self):
        return self.inputs.shape[0]

    def __getitem__(self, idx):
        x = torch.tensor(self.inputs[idx], dtype=torch.float32)
        y = torch.tensor(self.targets[idx], dtype=torch.float32)
        # 資料標準化映射至 [-1.0, 1.0]
        x = (x - 0.5) / 0.55
        y = (y - 0.5) / 0.55
        x = torch.clamp(x, -1.0, 1.0)
        y = torch.clamp(y, -1.0, 1.0)
        return x, y

DATA_DIR = "processed_dataset"
full_dataset = DeepSpaceDataset(DATA_DIR)

dataset_size = len(full_dataset)
test_size = int(0.2 * dataset_size)
train_size = dataset_size - test_size

train_dataset, test_dataset = random_split(full_dataset, [train_size, test_size],
generator=torch.Generator().manual_seed(42))
train_dataloader = DataLoader(train_dataset, batch_size=16, shuffle=True, drop_last=True)
test_dataloader = DataLoader(test_dataset, batch_size=16, shuffle=False, drop_last=False)

# =====
# 2. 神經網路架構
# =====
class UNetDown(nn.Module):
    def __init__(self, in_channels, out_channels, normalize=True, dropout=0.0):
        super().__init__()
        layers = [nn.Conv2d(in_channels, out_channels, kernel_size=4, stride=2, padding=1,
bias=False)]
        if normalize:
            layers.append(nn.BatchNorm2d(out_channels))
        layers.append(nn.LeakyReLU(0.2))
        if dropout > 0.0:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)
    def forward(self, x): return self.model(x)

```

```

class UNetUp(nn.Module):
    def __init__(self, in_channels, out_channels, dropout=0.0):
        super().__init__()
        layers = [
            nn.ConvTranspose2d(in_channels, out_channels, kernel_size=4, stride=2, padding=1,
bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        ]
        if dropout > 0.0:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)
    def forward(self, x, skip_input):
        x = self.model(x)
        return torch.cat((x, skip_input), 1)

```

```

class GeneratorUNet(nn.Module):
    def __init__(self, in_channels=1, out_channels=1):
        super().__init__()
        self.down1 = UNetDown(in_channels, 64, normalize=False)
        self.down2 = UNetDown(64, 128)
        self.down3 = UNetDown(128, 256)
        self.down4 = UNetDown(256, 512, dropout=0.5)
        self.down5 = UNetDown(512, 512, dropout=0.5)
        self.up1 = UNetUp(512, 512, dropout=0.5)
        self.up2 = UNetUp(1024, 256)
        self.up3 = UNetUp(512, 128)
        self.up4 = UNetUp(256, 64)
        self.final = nn.Sequential(nn.ConvTranspose2d(128, out_channels, kernel_size=4,
stride=2, padding=1), nn.Tanh())
    def forward(self, x):
        d1 = self.down1(x)
        d2 = self.down2(d1)
        d3 = self.down3(d2)
        d4 = self.down4(d3)
        d5 = self.down5(d4)
        u1 = self.up1(d5, d4)
        u2 = self.up2(u1, d3)
        u3 = self.up3(u2, d2)
        u4 = self.up4(u3, d1)
        return self.final(u4)

```

```

class Discriminator(nn.Module):
    def __init__(self, in_channels=1):

```

```

super().__init__()
def discriminator_block(in_filters, out_filters, normalization=True):
    layers = [nn.Conv2d(in_filters, out_filters, 4, stride=2, padding=1)]
    if normalization: layers.append(nn.BatchNorm2d(out_filters))
    layers.append(nn.LeakyReLU(0.2, inplace=True))
    return layers
self.model = nn.Sequential(
    *discriminator_block(in_channels * 2, 64, normalization=False),
    *discriminator_block(64, 128),
    *discriminator_block(128, 256),
    *discriminator_block(256, 512),
    nn.Conv2d(512, 1, 4, padding=1, bias=False)
)
def forward(self, img_A, img_B):
    img_input = torch.cat((img_A, img_B), 1)
    return self.model(img_input)

# =====
# 3. 物理約束損失函數定義
# =====
class PhysicsInformedLossV5(nn.Module):
    def __init__(self, lambda_pixel=100, lambda_fft_amp=5, lambda_phase=20,
lambda_flux=100, asinh_a=0.1):
        super().__init__()
        self.lambda_pixel = lambda_pixel
        self.lambda_fft_amp = lambda_fft_amp
        self.lambda_phase = lambda_phase
        self.lambda_flux = lambda_flux
        self.criterion_mse = nn.MSELoss()
        self.criterion_l1 = nn.L1Loss()
        self.a = asinh_a
        self.arcsinh_1_over_a = np.arcsinh(1.0 / self.a)

    def inverse_transform_to_linear(self, tensor_img):
        y_original = (tensor_img * 0.55) + 0.5
        y_clamped = torch.clamp(y_original, 0.0, 1.0)
        return self.a * torch.sinh(y_clamped * self.arcsinh_1_over_a)

    def forward(self, generated, target):
        gen_linear = self.inverse_transform_to_linear(generated)
        target_linear = self.inverse_transform_to_linear(target)

        loss_pixel = self.criterion_mse(generated, target)

```

```

fft_gen = torch.fft.fft2(generated, norm="ortho")
fft_target = torch.fft.fft2(target, norm="ortho")

loss_fft_amp = self.criterion_l1(torch.abs(fft_gen), torch.abs(fft_target))
loss_phase = self.criterion_mse(torch.angle(fft_gen), torch.angle(fft_target))
loss_fft = (self.lambda_fft_amp * loss_fft_amp) + (self.lambda_phase * loss_phase)

star_mask = (target > 0.0).float().detach()
flux_gen = torch.sum(gen_linear * star_mask, dim=(2, 3))
flux_target = torch.sum(target_linear * star_mask, dim=(2, 3))
loss_flux_mape = torch.mean(torch.abs(flux_gen - flux_target) / (flux_target + 1e-8))

total_pi_loss = (self.lambda_pixel * loss_pixel) + loss_fft + (self.lambda_flux *
loss_flux_mape)
return total_pi_loss, loss_flux_mape, loss_phase

# =====
# 4. 微調環境設定與權重載入
# =====
OUTPUT_V5_DIR = "PI_cGAN_V5_Target_results"
os.makedirs(OUTPUT_V5_DIR, exist_ok=True)
print(f"已建立存檔資料夾：{OUTPUT_V5_DIR}")

generator_v5 = GeneratorUNet().to(device)
discriminator_v5 = Discriminator().to(device)

BEST_V4_WEIGHTS = "PI_cGAN_V4_Finetune_results/v4_best_finetuned_weights.pth"
generator_v5.load_state_dict(torch.load(BEST_V4_WEIGHTS, map_location=device))

LR_V5 = 0.00001
optimizer_G_v5 = optim.Adam(generator_v5.parameters(), lr=LR_V5, betas=(0.5, 0.999))
optimizer_D_v5 = optim.Adam(discriminator_v5.parameters(), lr=LR_V5, betas=(0.5,
0.999))

pi_criterion_v5 = PhysicsInformedLossV5().to(device)
criterion_GAN_v5 = nn.BCEWithLogitsLoss().to(device)

# =====
# 5. 微調訓練迴圈
# =====
epochs_v5 = 50
best_val_ssim = 0.0

```

```

patience = 15
no_improve_count = 0

for epoch in range(epochs_v5):
    generator_v5.train()
    discriminator_v5.train()

    epoch_ms_ssim_loss = 0.0
    epoch_flux_mape_val = 0.0
    epoch_phase_mse_val = 0.0

    for i, (real_A, real_B) in enumerate(train_dataloader):
        real_A, real_B = real_A.to(device), real_B.to(device)

        optimizer_G_v5.zero_grad()
        fake_B = generator_v5(real_A)

        pred_fake = discriminator_v5(real_A, fake_B)
        loss_GAN = criterion_GAN_v5(pred_fake, torch.ones_like(pred_fake, device=device))

        total_pi_loss, flux_mape, phase_mse = pi_criterion_v5(fake_B, real_B)

        gen_01 = (fake_B + 1.0) / 2.0
        real_01 = (real_B + 1.0) / 2.0
        loss_ms_ssim = 1 - ms_ssim(gen_01, real_01, data_range=1.0, size_average=True)

        loss_G = loss_GAN + total_pi_loss + (15 * loss_ms_ssim)
        loss_G.backward()
        optimizer_G_v5.step()

        epoch_ms_ssim_loss += loss_ms_ssim.item()
        epoch_flux_mape_val += flux_mape.item()
        epoch_phase_mse_val += phase_mse.item()

        optimizer_D_v5.zero_grad()
        pred_real = discriminator_v5(real_A, real_B)
        loss_real = criterion_GAN_v5(pred_real, torch.ones_like(pred_real))
        pred_fake_detach = discriminator_v5(real_A, fake_B.detach())
        loss_fake = criterion_GAN_v5(pred_fake_detach, torch.zeros_like(pred_fake_detach))
        loss_D = 0.5 * (loss_real + loss_fake)
        loss_D.backward()
        optimizer_D_v5.step()

    generator_v5.eval()

```



```

val_ssim_list = []

with torch.no_grad():
    for test_A, test_B in test_dataloader:
        test_A, test_B = test_A.to(device), test_B.to(device)
        test_fake_B = generator_v5(test_A)

        np_fake = test_fake_B.cpu().numpy()
        np_real = test_B.cpu().numpy()
        for b in range(np_fake.shape[0]):
            val_ssim_list.append(ssim_metric(np_real[b, 0], np_fake[b, 0], data_range=2.0))

    current_epoch_ssim = np.mean(val_ssim_list)
    avg_flux = epoch_flux_mape_val / len(train_dataloader)
    avg_phase = epoch_phase_mse_val / len(train_dataloader)

    print(f"Epoch [{epoch+1}/{epochs_v5}] | Val SSIM: {current_epoch_ssim:.4f} | Flux
Error: {avg_flux*100:.2f}% | Phase MSE: {avg_phase:.4f}")

    if current_epoch_ssim > best_val_ssim:
        best_val_ssim = current_epoch_ssim
        no_improve_count = 0

        torch.save(generator_v5.state_dict(), os.path.join(OUTPUT_V5_DIR,
"v5_best_finetuned_weights.pth"))

        test_A_sample, test_B_sample = next(iter(test_dataloader))
        test_fake_sample = generator_v5(test_A_sample.to(device))
        np.savez_compressed(os.path.join(OUTPUT_V5_DIR, "v5_best_test_data.npz"),
            inputs=test_A_sample.cpu().numpy(),
            targets=test_B_sample.cpu().numpy(),
            generated=test_fake_sample.detach().cpu().numpy())
    else:
        no_improve_count += 1

    if no_improve_count >= patience:
        break

print("PI-cGAN V5 訓練結束。")

```

8.PI-cGAN V4.1:

```

import os
import glob
import numpy as np

```

```

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split
import matplotlib.pyplot as plt
from IPython.display import clear_output
from tqdm.notebook import tqdm
from skimage.metrics import structural_similarity as ssim_metric

try:
    from pytorch_msssim import ms_ssim
except ImportError:
    raise ImportError("請先安裝 pytorch-msssim")

# =====
# 1. 環境設定與資料載入
# =====
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"運算裝置: {device}")

class DeepSpaceDataset(Dataset):
    def __init__(self, npz_folder):
        self.inputs = []
        self.targets = []
        file_paths = glob.glob(os.path.join(npz_folder, "*.npz"))
        for f in file_paths:
            data = np.load(f)
            self.inputs.append(data['inputs'])
            self.targets.append(data['targets'])
        self.inputs = np.concatenate(self.inputs, axis=0)
        self.targets = np.concatenate(self.targets, axis=0)
        self.inputs = np.expand_dims(self.inputs, axis=1)
        self.targets = np.expand_dims(self.targets, axis=1)

    def __len__(self):
        return self.inputs.shape[0]

    def __getitem__(self, idx):
        x = torch.tensor(self.inputs[idx], dtype=torch.float32)
        y = torch.tensor(self.targets[idx], dtype=torch.float32)
        x = (x - 0.5) / 0.55
        y = (y - 0.5) / 0.55
        x = torch.clamp(x, -1.0, 1.0)

```

```

        y = torch.clamp(y, -1.0, 1.0)
        return x, y

DATA_DIR = "processed_dataset"
full_dataset = DeepSpaceDataset(DATA_DIR)

dataset_size = len(full_dataset)
test_size = int(0.2 * dataset_size)
train_size = dataset_size - test_size

train_dataset, test_dataset = random_split(full_dataset, [train_size, test_size],
generator=torch.Generator().manual_seed(42))
train_dataloader = DataLoader(train_dataset, batch_size=16, shuffle=True, drop_last=True)
test_dataloader = DataLoader(test_dataset, batch_size=16, shuffle=False, drop_last=False)

# =====
# 2. 神經網路架構
# =====
class UNetDown(nn.Module):
    def __init__(self, in_channels, out_channels, normalize=True, dropout=0.0):
        super().__init__()
        layers = [nn.Conv2d(in_channels, out_channels, kernel_size=4, stride=2, padding=1,
bias=False)]
        if normalize:
            layers.append(nn.BatchNorm2d(out_channels))
        layers.append(nn.LeakyReLU(0.2))
        if dropout > 0.0:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)
    def forward(self, x): return self.model(x)

class UNetUp(nn.Module):
    def __init__(self, in_channels, out_channels, dropout=0.0):
        super().__init__()
        layers = [
            nn.ConvTranspose2d(in_channels, out_channels, kernel_size=4, stride=2, padding=1,
bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        ]
        if dropout > 0.0:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)

```

```

def forward(self, x, skip_input):
    x = self.model(x)
    return torch.cat((x, skip_input), 1)

class GeneratorUNet(nn.Module):
    def __init__(self, in_channels=1, out_channels=1):
        super().__init__()
        self.down1 = UNetDown(in_channels, 64, normalize=False)
        self.down2 = UNetDown(64, 128)
        self.down3 = UNetDown(128, 256)
        self.down4 = UNetDown(256, 512, dropout=0.5)
        self.down5 = UNetDown(512, 512, dropout=0.5)
        self.up1 = UNetUp(512, 512, dropout=0.5)
        self.up2 = UNetUp(1024, 256)
        self.up3 = UNetUp(512, 128)
        self.up4 = UNetUp(256, 64)
        self.final = nn.Sequential(nn.ConvTranspose2d(128, out_channels, kernel_size=4,
stride=2, padding=1), nn.Tanh())
    def forward(self, x):
        d1 = self.down1(x)
        d2 = self.down2(d1)
        d3 = self.down3(d2)
        d4 = self.down4(d3)
        d5 = self.down5(d4)
        u1 = self.up1(d5, d4)
        u2 = self.up2(u1, d3)
        u3 = self.up3(u2, d2)
        u4 = self.up4(u3, d1)
        return self.final(u4)

class Discriminator(nn.Module):
    def __init__(self, in_channels=1):
        super().__init__()
        def discriminator_block(in_filters, out_filters, normalization=True):
            layers = [nn.Conv2d(in_filters, out_filters, 4, stride=2, padding=1)]
            if normalization: layers.append(nn.BatchNorm2d(out_filters))
            layers.append(nn.LeakyReLU(0.2, inplace=True))
            return layers
        self.model = nn.Sequential(
            *discriminator_block(in_channels * 2, 64, normalization=False),
            *discriminator_block(64, 128),
            *discriminator_block(128, 256),
            *discriminator_block(256, 512),
            nn.Conv2d(512, 1, 4, padding=1, bias=False)

```

```

    )
    def forward(self, img_A, img_B):
        img_input = torch.cat((img_A, img_B), 1)
        return self.model(img_input)

# =====
# 3. 物理資訊損失函數
# =====

class PhysicsInformedLossV3(nn.Module):
    def __init__(self, lambda_pixel=100, lambda_fft=0.1, lambda_flux=0.5, asinh_a=0.1):
        super().__init__()
        self.lambda_pixel = lambda_pixel
        self.lambda_fft = lambda_fft
        self.lambda_flux = lambda_flux
        self.criterion_l1 = nn.L1Loss()
        self.a = asinh_a
        self.arcsinh_1_over_a = np.arcsinh(1.0 / self.a)

    def inverse_transform_to_linear(self, tensor_img):
        y_original = (tensor_img * 0.55) + 0.5
        y_clamped = torch.clamp(y_original, 0.0, 1.0)
        return self.a * torch.sinh(y_clamped * self.arcsinh_1_over_a)

    def forward(self, generated, target):
        gen_linear = self.inverse_transform_to_linear(generated)
        target_linear = self.inverse_transform_to_linear(target)

        loss_pixel = self.criterion_l1(gen_linear, target_linear)

        fft_gen = torch.fft.fft2(generated, norm="ortho")
        fft_target = torch.fft.fft2(target, norm="ortho")
        loss_fft_real = self.criterion_l1(torch.real(fft_gen), torch.real(fft_target))
        loss_fft_imag = self.criterion_l1(torch.imag(fft_gen), torch.imag(fft_target))
        loss_fft = loss_fft_real + loss_fft_imag

        star_mask = (target > 0.0).float().detach()
        flux_gen = torch.sum(gen_linear * star_mask, dim=(2, 3))
        flux_target = torch.sum(target_linear * star_mask, dim=(2, 3))
        loss_flux = self.criterion_l1(flux_gen, flux_target)

        total_pi_loss = (self.lambda_pixel * loss_pixel) + (self.lambda_fft * loss_fft) +
        (self.lambda_flux * loss_flux)
        return total_pi_loss, loss_pixel, loss_fft, loss_flux

```

```

# =====
# 4. 模型初始化與參數設定
# =====
OUTPUT_V4_1_DIR = "PI_cGAN_V4_1_Balanced_results"
os.makedirs(OUTPUT_V4_1_DIR, exist_ok=True)
print(f"資料儲存目錄: {OUTPUT_V4_1_DIR}")

generator_v4 = GeneratorUNet().to(device)
discriminator_v4 = Discriminator().to(device)

BEST_V3_WEIGHTS = "PI_cGAN_V3_Ultimate_results/picgan_ep120_weights.pth"
print(f"載入權重: {BEST_V3_WEIGHTS}")
generator_v4.load_state_dict(torch.load(BEST_V3_WEIGHTS, map_location=device))

LR_V4 = 0.00002
optimizer_G_v4 = optim.Adam(generator_v4.parameters(), lr=LR_V4, betas=(0.5, 0.999))
optimizer_D_v4 = optim.Adam(discriminator_v4.parameters(), lr=LR_V4, betas=(0.5,
0.999))

pi_criterion_v4 = PhysicsInformedLossV3().to(device)
criterion_GAN_v4 = nn.BCEWithLogitsLoss().to(device)

# =====
# 5. 訓練迴圈
# =====
epochs_v4 = 100
best_val_ssim = 0.0
patience = 20
no_improve_count = 0

for epoch in range(epochs_v4):
    generator_v4.train()
    discriminator_v4.train()
    epoch_ms_ssim_loss = 0.0

    for i, (real_A, real_B) in enumerate(train_dataloader):
        real_A, real_B = real_A.to(device), real_B.to(device)

        optimizer_G_v4.zero_grad()
        fake_B = generator_v4(real_A)

        pred_fake = discriminator_v4(real_A, fake_B)

```

```

loss_GAN = criterion_GAN_v4(pred_fake, torch.ones_like(pred_fake, device=device))

total_pi_loss, _, _, _ = pi_criterion_v4(fake_B, real_B)

gen_01 = (fake_B + 1.0) / 2.0
real_01 = (real_B + 1.0) / 2.0
loss_ms_ssim = 1 - ms_ssim(gen_01, real_01, data_range=1.0, size_average=True)

loss_G = loss_GAN + total_pi_loss + (15 * loss_ms_ssim)
loss_G.backward()
optimizer_G_v4.step()

epoch_ms_ssim_loss += loss_ms_ssim.item()

optimizer_D_v4.zero_grad()
pred_real = discriminator_v4(real_A, real_B)
loss_real = criterion_GAN_v4(pred_real, torch.ones_like(pred_real))
pred_fake_detach = discriminator_v4(real_A, fake_B.detach())
loss_fake = criterion_GAN_v4(pred_fake_detach, torch.zeros_like(pred_fake_detach))
loss_D = 0.5 * (loss_real + loss_fake)
loss_D.backward()
optimizer_D_v4.step()

generator_v4.eval()
val_ssim_list = []

with torch.no_grad():
    for test_A, test_B in test_dataloader:
        test_A, test_B = test_A.to(device), test_B.to(device)
        test_fake_B = generator_v4(test_A)

        np_fake = test_fake_B.cpu().numpy()
        np_real = test_B.cpu().numpy()
        for b in range(np_fake.shape[0]):
            val_ssim_list.append(ssim_metric(np_real[b, 0], np_fake[b, 0], data_range=2.0))

current_epoch_ssim = np.mean(val_ssim_list)

print(f"Epoch [{epoch+1}/{epochs_v4}] | MS-SSIM Loss:
{epoch_ms_ssim_loss/len(train_dataloader):.4f} | Validation SSIM:
{current_epoch_ssim:.4f}")

if current_epoch_ssim > best_val_ssim:
    best_val_ssim = current_epoch_ssim

```

```

no_improve_count = 0

best_pth_path = os.path.join(OUTPUT_V4_1_DIR,
"v4_1_best_finetuned_weights.pth")
torch.save(generator_v4.state_dict(), best_pth_path)

best_npz_path = os.path.join(OUTPUT_V4_1_DIR, "v4_1_best_test_data.npz")
test_A_sample, test_B_sample = next(iter(test_dataloader))
test_fake_sample = generator_v4(test_A_sample.to(device))
np.savez_compressed(best_npz_path,
                    inputs=test_A_sample.cpu().numpy(),
                    targets=test_B_sample.cpu().numpy(),
                    generated=test_fake_sample.detach().cpu().numpy())

print(f"紀錄更新：最佳權重與測試資料已儲存至 {OUTPUT_V4_1_DIR}")
else:
    no_improve_count += 1
    print(f"SSIM 未提升 (最佳: {best_val_ssim:.4f})，早停計數:
{no_improve_count}/{patience}")

    if no_improve_count >= patience:
        print(f"觸發早停，訓練結束。")
        break

```

V4.1 第二段：

```

import os
import glob
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split
import matplotlib.pyplot as plt
from IPython.display import clear_output
from tqdm.notebook import tqdm
from skimage.metrics import structural_similarity as ssim_metric

try:
    from pytorch_msssim import ms_ssim
except ImportError:
    raise ImportError("請先安裝 pytorch-msssim")

# =====

```


1. 環境設定與資料載入

=====

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print(f"目前使用的運算裝置: {device}")

```
class DeepSpaceDataset(Dataset):
```

```
    def __init__(self, npz_folder):
```

```
        self.inputs = []
```

```
        self.targets = []
```

```
        file_paths = glob.glob(os.path.join(npz_folder, "*.npz"))
```

```
        for f in file_paths:
```

```
            data = np.load(f)
```

```
            self.inputs.append(data['inputs'])
```

```
            self.targets.append(data['targets'])
```

```
        self.inputs = np.concatenate(self.inputs, axis=0)
```

```
        self.targets = np.concatenate(self.targets, axis=0)
```

```
        self.inputs = np.expand_dims(self.inputs, axis=1)
```

```
        self.targets = np.expand_dims(self.targets, axis=1)
```

```
    def __len__(self):
```

```
        return self.inputs.shape[0]
```

```
    def __getitem__(self, idx):
```

```
        x = torch.tensor(self.inputs[idx], dtype=torch.float32)
```

```
        y = torch.tensor(self.targets[idx], dtype=torch.float32)
```

```
        x = (x - 0.5) / 0.55
```

```
        y = (y - 0.5) / 0.55
```

```
        x = torch.clamp(x, -1.0, 1.0)
```

```
        y = torch.clamp(y, -1.0, 1.0)
```

```
        return x, y
```

```
DATA_DIR = "processed_dataset"
```

```
full_dataset = DeepSpaceDataset(DATA_DIR)
```

```
dataset_size = len(full_dataset)
```

```
test_size = int(0.2 * dataset_size)
```

```
train_size = dataset_size - test_size
```

```
train_dataset, test_dataset = random_split(full_dataset, [train_size, test_size],
```

```
generator=torch.Generator().manual_seed(42))
```

```
train_dataloader = DataLoader(train_dataset, batch_size=16, shuffle=True, drop_last=True)
```

```
test_dataloader = DataLoader(test_dataset, batch_size=16, shuffle=False, drop_last=False)
```

```

# =====
# 2. 神經網路架構
# =====
class UNetDown(nn.Module):
    def __init__(self, in_channels, out_channels, normalize=True, dropout=0.0):
        super().__init__()
        layers = [nn.Conv2d(in_channels, out_channels, kernel_size=4, stride=2, padding=1,
bias=False)]
        if normalize:
            layers.append(nn.BatchNorm2d(out_channels))
        layers.append(nn.LeakyReLU(0.2))
        if dropout > 0.0:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)
    def forward(self, x): return self.model(x)

class UNetUp(nn.Module):
    def __init__(self, in_channels, out_channels, dropout=0.0):
        super().__init__()
        layers = [
            nn.ConvTranspose2d(in_channels, out_channels, kernel_size=4, stride=2, padding=1,
bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        ]
        if dropout > 0.0:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)
    def forward(self, x, skip_input):
        x = self.model(x)
        return torch.cat((x, skip_input), 1)

class GeneratorUNet(nn.Module):
    def __init__(self, in_channels=1, out_channels=1):
        super().__init__()
        self.down1 = UNetDown(in_channels, 64, normalize=False)
        self.down2 = UNetDown(64, 128)
        self.down3 = UNetDown(128, 256)
        self.down4 = UNetDown(256, 512, dropout=0.5)
        self.down5 = UNetDown(512, 512, dropout=0.5)
        self.up1 = UNetUp(512, 512, dropout=0.5)
        self.up2 = UNetUp(1024, 256)
        self.up3 = UNetUp(512, 128)

```

```

        self.up4 = UNetUp(256, 64)
        self.final = nn.Sequential(nn.ConvTranspose2d(128, out_channels, kernel_size=4,
stride=2, padding=1), nn.Tanh())
    def forward(self, x):
        d1 = self.down1(x)
        d2 = self.down2(d1)
        d3 = self.down3(d2)
        d4 = self.down4(d3)
        d5 = self.down5(d4)
        u1 = self.up1(d5, d4)
        u2 = self.up2(u1, d3)
        u3 = self.up3(u2, d2)
        u4 = self.up4(u3, d1)
        return self.final(u4)

class Discriminator(nn.Module):
    def __init__(self, in_channels=1):
        super().__init__()
        def discriminator_block(in_filters, out_filters, normalization=True):
            layers = [nn.Conv2d(in_filters, out_filters, 4, stride=2, padding=1)]
            if normalization: layers.append(nn.BatchNorm2d(out_filters))
            layers.append(nn.LeakyReLU(0.2, inplace=True))
            return layers
        self.model = nn.Sequential(
            *discriminator_block(in_channels * 2, 64, normalization=False),
            *discriminator_block(64, 128),
            *discriminator_block(128, 256),
            *discriminator_block(256, 512),
            nn.Conv2d(512, 1, 4, padding=1, bias=False)
        )
    def forward(self, img_A, img_B):
        img_input = torch.cat((img_A, img_B), 1)
        return self.model(img_input)

# =====
# 3. 物理資訊損失函數
# =====
class PhysicsInformedLossV3(nn.Module):
    def __init__(self, lambda_pixel=100, lambda_fft=0.1, lambda_flux=0.5, asinh_a=0.1):
        super().__init__()
        self.lambda_pixel = lambda_pixel
        self.lambda_fft = lambda_fft
        self.lambda_flux = lambda_flux

```

```

self.criterion_l1 = nn.L1Loss()
self.a = asinh_a
self.arcsinh_1_over_a = np.arcsinh(1.0 / self.a)

def inverse_transform_to_linear(self, tensor_img):
    y_original = (tensor_img * 0.55) + 0.5
    y_clamped = torch.clamp(y_original, 0.0, 1.0)
    return self.a * torch.sinh(y_clamped * self.arcsinh_1_over_a)

def forward(self, generated, target):
    gen_linear = self.inverse_transform_to_linear(generated)
    target_linear = self.inverse_transform_to_linear(target)

    loss_pixel = self.criterion_l1(gen_linear, target_linear)

    fft_gen = torch.fft.fft2(generated, norm="ortho")
    fft_target = torch.fft.fft2(target, norm="ortho")
    loss_fft_real = self.criterion_l1(torch.real(fft_gen), torch.real(fft_target))
    loss_fft_imag = self.criterion_l1(torch.imag(fft_gen), torch.imag(fft_target))
    loss_fft = loss_fft_real + loss_fft_imag

    star_mask = (target > 0.0).float().detach()
    flux_gen = torch.sum(gen_linear * star_mask, dim=(2, 3))
    flux_target = torch.sum(target_linear * star_mask, dim=(2, 3))
    loss_flux = self.criterion_l1(flux_gen, flux_target)

    total_pi_loss = (self.lambda_pixel * loss_pixel) + (self.lambda_fft * loss_fft) +
    (self.lambda_flux * loss_flux)
    return total_pi_loss, loss_pixel, loss_fft, loss_flux

# =====
# 4. 模型初始化與參數設定
# =====
OUTPUT_PHASE2_DIR = "PI_cGAN_V4_1_Phase2_results"
os.makedirs(OUTPUT_PHASE2_DIR, exist_ok=True)
print(f"資料儲存目錄: {OUTPUT_PHASE2_DIR}")

generator_v4 = GeneratorUNet().to(device)
discriminator_v4 = Discriminator().to(device)

PREVIOUS_BEST_WEIGHTS =
"PI_cGAN_V4_1_Balanced_results/v4_1_best_finetuned_weights.pth"
if os.path.exists(PREVIOUS_BEST_WEIGHTS):

```

```

print(f"載入權重: {PREVIOUS_BEST_WEIGHTS}")
generator_v4.load_state_dict(torch.load(PREVIOUS_BEST_WEIGHTS,
map_location=device))
else:
    print(f"警告: 找不到檔案 {PREVIOUS_BEST_WEIGHTS}")

LR_V4 = 0.00002
optimizer_G_v4 = optim.Adam(generator_v4.parameters(), lr=LR_V4, betas=(0.5, 0.999))
optimizer_D_v4 = optim.Adam(discriminator_v4.parameters(), lr=LR_V4, betas=(0.5,
0.999))

pi_criterion_v4 = PhysicsInformedLossV3().to(device)
criterion_GAN_v4 = nn.BCEWithLogitsLoss().to(device)

# =====
# 5. 訓練迴圈
# =====
epochs_v4 = 100
best_val_ssim = 0.0
patience = 30
no_improve_count = 0

for epoch in range(epochs_v4):
    generator_v4.train()
    discriminator_v4.train()
    epoch_ms_ssim_loss = 0.0

    for i, (real_A, real_B) in enumerate(train_dataloader):
        real_A, real_B = real_A.to(device), real_B.to(device)

        optimizer_G_v4.zero_grad()
        fake_B = generator_v4(real_A)

        pred_fake = discriminator_v4(real_A, fake_B)
        loss_GAN = criterion_GAN_v4(pred_fake, torch.ones_like(pred_fake, device=device))

        total_pi_loss, _, _, _ = pi_criterion_v4(fake_B, real_B)

        gen_01 = (fake_B + 1.0) / 2.0
        real_01 = (real_B + 1.0) / 2.0
        loss_ms_ssim = 1 - ms_ssim(gen_01, real_01, data_range=1.0, size_average=True)

        loss_G = loss_GAN + total_pi_loss + (15 * loss_ms_ssim)

```

```

loss_G.backward()
optimizer_G_v4.step()

epoch_ms_ssim_loss += loss_ms_ssim.item()

optimizer_D_v4.zero_grad()
pred_real = discriminator_v4(real_A, real_B)
loss_real = criterion_GAN_v4(pred_real, torch.ones_like(pred_real))
pred_fake_detach = discriminator_v4(real_A, fake_B.detach())
loss_fake = criterion_GAN_v4(pred_fake_detach, torch.zeros_like(pred_fake_detach))
loss_D = 0.5 * (loss_real + loss_fake)
loss_D.backward()
optimizer_D_v4.step()

generator_v4.eval()
val_ssim_list = []

with torch.no_grad():
    for test_A, test_B in test_dataloader:
        test_A, test_B = test_A.to(device), test_B.to(device)
        test_fake_B = generator_v4(test_A)

        np_fake = test_fake_B.cpu().numpy()
        np_real = test_B.cpu().numpy()
        for b in range(np_fake.shape[0]):
            val_ssim_list.append(ssim_metric(np_real[b, 0], np_fake[b, 0], data_range=2.0))

current_epoch_ssim = np.mean(val_ssim_list)

print(f"Epoch [{epoch+101}/{epochs_v4+100}] | MS-SSIM Loss:
{epoch_ms_ssim_loss/len(train_dataloader):.4f} | Validation SSIM:
{current_epoch_ssim:.4f}")

if current_epoch_ssim > best_val_ssim:
    best_val_ssim = current_epoch_ssim
    no_improve_count = 0

    best_pth_path = os.path.join(OUTPUT_PHASE2_DIR,
"v4_1_phase2_best_weights.pth")
    torch.save(generator_v4.state_dict(), best_pth_path)

    best_npz_path = os.path.join(OUTPUT_PHASE2_DIR,
"v4_1_phase2_best_test_data.npz")
    test_A_sample, test_B_sample = next(iter(test_dataloader))

```

```

test_fake_sample = generator_v4(test_A_sample.to(device))
np.savez_compressed(best_npz_path,
                    inputs=test_A_sample.cpu().numpy(),
                    targets=test_B_sample.cpu().numpy(),
                    generated=test_fake_sample.detach().cpu().numpy())

print(f"紀錄更新：最佳參數與測試資料已儲存至 {OUTPUT_PHASE2_DIR}")
else:
    no_improve_count += 1
    print(f"SSIM 未提升 (目前最佳: {best_val_ssim:.4f})，早停計數:
{no_improve_count}/{patience}")

if no_improve_count >= patience:
    print(f"早停觸發，訓練結束。")
    break

```

9.PI-cGAN V1-V3 評估腳本:

```

import os
import glob
import re
import torch
import numpy as np
import matplotlib.pyplot as plt
from torch.utils.data import Dataset, DataLoader, random_split
from skimage.metrics import structural_similarity as ssim
from tqdm.notebook import tqdm

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"目前使用的運算裝置: {device}")

# =====
# 1. 資料處理類別
# =====
class DeepSpaceDataset(Dataset):
    def __init__(self, npz_folder):
        self.inputs = []
        self.targets = []
        file_paths = glob.glob(os.path.join(npz_folder, "*.npz"))
        for f in file_paths:
            data = np.load(f)
            self.inputs.append(data['inputs'])
            self.targets.append(data['targets'])

```

```

        self.inputs = np.concatenate(self.inputs, axis=0)
        self.targets = np.concatenate(self.targets, axis=0)
        self.inputs = np.expand_dims(self.inputs, axis=1)
        self.targets = np.expand_dims(self.targets, axis=1)

    def __len__(self):
        return self.inputs.shape[0]

    def __getitem__(self, idx):
        x = torch.tensor(self.inputs[idx], dtype=torch.float32)
        y = torch.tensor(self.targets[idx], dtype=torch.float32)

        x = (x - 0.5) / 0.55
        y = (y - 0.5) / 0.55
        x = torch.clamp(x, -1.0, 1.0)
        y = torch.clamp(y, -1.0, 1.0)
        return x, y

DATA_DIR = "processed_dataset"
full_dataset = DeepSpaceDataset(DATA_DIR)

dataset_size = len(full_dataset)
test_size = int(0.2 * dataset_size)
train_size = dataset_size - test_size

train_dataset, test_dataset = random_split(full_dataset, [train_size, test_size],
generator=torch.Generator().manual_seed(42))
test_dataloader = DataLoader(test_dataset, batch_size=16, shuffle=False, drop_last=False)

print(f"測試集數量: {test_size}")

# =====
# 2. 神經網路架構定義
# =====
class UNetDown(nn.Module):
    def __init__(self, in_channels, out_channels, normalize=True, dropout=0.0):
        super().__init__()
        layers = [nn.Conv2d(in_channels, out_channels, kernel_size=4, stride=2, padding=1,
bias=False)]
        if normalize:
            layers.append(nn.BatchNorm2d(out_channels))
        layers.append(nn.LeakyReLU(0.2))
        if dropout > 0.0:

```



```

        layers.append(nn.Dropout(dropout))
    self.model = nn.Sequential(*layers)
    def forward(self, x):
        return self.model(x)

class UNetUp(nn.Module):
    def __init__(self, in_channels, out_channels, dropout=0.0):
        super().__init__()
        layers = [
            nn.ConvTranspose2d(in_channels, out_channels, kernel_size=4, stride=2, padding=1,
bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        ]
        if dropout > 0.0:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)
    def forward(self, x, skip_input):
        x = self.model(x)
        x = torch.cat((x, skip_input), 1)
        return x

class GeneratorUNet(nn.Module):
    def __init__(self, in_channels=1, out_channels=1):
        super().__init__()
        self.down1 = UNetDown(in_channels, 64, normalize=False)
        self.down2 = UNetDown(64, 128)
        self.down3 = UNetDown(128, 256)
        self.down4 = UNetDown(256, 512, dropout=0.5)
        self.down5 = UNetDown(512, 512, dropout=0.5)
        self.up1 = UNetUp(512, 512, dropout=0.5)
        self.up2 = UNetUp(1024, 256)
        self.up3 = UNetUp(512, 128)
        self.up4 = UNetUp(256, 64)
        self.final = nn.Sequential(nn.ConvTranspose2d(128, out_channels, kernel_size=4,
stride=2, padding=1), nn.Tanh())
    def forward(self, x):
        d1 = self.down1(x)
        d2 = self.down2(d1)
        d3 = self.down3(d2)
        d4 = self.down4(d3)
        d5 = self.down5(d4)
        u1 = self.up1(d5, d4)
        u2 = self.up2(u1, d3)

```

```

        u3 = self.up3(u2, d2)
        u4 = self.up4(u3, d1)
        return self.final(u4)

# =====
# 3. 評估程序
# =====
def inverse_to_linear_numpy(img, a=0.1):
    y_original = (img * 0.55) + 0.5
    y_clamped = np.clip(y_original, 0.0, 1.0)
    linear_img = a * np.sinh(y_clamped * np.arcsinh(1.0 / a))
    return linear_img

weight_dir = "PI_cGAN_V2_Optimized_results"
weight_files = glob.glob(os.path.join(weight_dir, "*.pth"))
weight_files.sort(key=lambda f: int(re.search(r'ep(\d+)', f).group(1)))

if not weight_files:
    print("錯誤: 找不到權重檔案。")
else:
    print(f"找到 {len(weight_files)} 個權重檔案。")

epochs_list = []
ssim_scores = []
flux_errors = []

eval_net = GeneratorUNet().to(device)
eval_net.eval()

with torch.no_grad():
    for weight_path in tqdm(weight_files, desc="執行評估"):
        ep_num = int(re.search(r'ep(\d+)', weight_path).group(1))
        epochs_list.append(ep_num)

        eval_net.load_state_dict(torch.load(weight_path, map_location=device,
        weights_only=True))

        epoch_ssim = 0.0
        epoch_flux_err = 0.0
        num_batches = 0

        for test_A, test_B in test_dataloader:
            test_A, test_B = test_A.to(device), test_B.to(device)

```

```

fake_B = eval_net(test_A)

np_fake = fake_B.cpu().numpy()
np_target = test_B.cpu().numpy()

batch_size = np_target.shape[0]
num_batches += batch_size

for b in range(batch_size):
    img_f = np_fake[b, 0]
    img_t = np_target[b, 0]

    val_ssim = ssim(img_t, img_f, data_range=2.0)
    epoch_ssim += val_ssim

    linear_f = inverse_to_linear_numpy(img_f)
    linear_t = inverse_to_linear_numpy(img_t)

    star_mask = (img_t > 0.0).astype(float)
    flux_f = np.sum(linear_f * star_mask)
    flux_t = np.sum(linear_t * star_mask)

    if flux_t > 1e-6:
        err = np.abs(flux_f - flux_t) / flux_t
    else:
        err = 0.0
    epoch_flux_err += err

ssim_scores.append(epoch_ssim / num_batches)
flux_errors.append((epoch_flux_err / num_batches) * 100)

# =====
# 4. 繪製評估圖表
# =====
plt.figure(figsize=(12, 5))

ax1 = plt.subplot(1, 2, 1)
ax1.plot(epochs_list, ssim_scores, marker='o', color='b', linestyle='-')
ax1.set_title("SSIM Score")
ax1.set_xlabel("Epoch")
ax1.set_ylabel("SSIM")
ax1.grid(True)

```

```

ax2 = plt.subplot(1, 2, 2)
ax2.plot(epochs_list, flux_errors, marker='s', color='r', linestyle='-')
ax2.set_title("Flux Error (%)")
ax2.set_xlabel("Epoch")
ax2.set_ylabel("Error (%)")
ax2.grid(True)

plt.tight_layout()
plt.show()

```

```

print("\n 評估報表：")
print("-" * 40)
print(f"{'Epoch':<10} | {'SSIM':<12} | {'Flux Error':<12}")
print("-" * 40)
for ep, s, f_err in zip(epochs_list, ssim_scores, flux_errors):
    print(f"{'ep':<10} | {'s':.4f} | {'f_err':.2f}%")

```

10. 四大指標圖表與數據產出腳本(以 V4 為例，其他亦可通用):

```

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from skimage.metrics import peak_signal_noise_ratio as psnr
from skimage.metrics import structural_similarity as ssim
from IPython.display import display

# =====
# 1. 資料讀取
# =====
file_path = "PI_cGAN_V4_Finetune_results/v4_best_test_data.npz"
data = np.load(file_path)

inputs = data['inputs']
targets = data['targets']
generated = data['generated']

num_images, _, h, w = inputs.shape
print(f"載入 {num_images} 張圖片進行分析。")

# 預先計算半徑矩陣用於 RAPS 計算
center = (int(h/2), int(w/2))
y, x = np.indices((h, w))
r_matrix = np.sqrt((x - center[1])**2 + (y - center[0])**2).astype(int)
nr = np.bincount(r_matrix.ravel())

```

```
def calculate_raps_fast(image):
    f_transform = np.fft.fft2(image)
    f_shift = np.fft.fftshift(f_transform)
    power_spectrum = np.abs(f_shift)**2
    tbin = np.bincount(r_matrix.ravel(), power_spectrum.ravel())
    return tbin / nr
```

```
def calculate_phase(image):
    f_transform = np.fft.fft2(image)
    f_shift = np.fft.fftshift(f_transform)
    return np.angle(f_shift)
```

```
# =====
```

```
# 2. 指標計算與物理空間守恆分析
```

```
# =====
```

```
all_raps_in, all_raps_gt, all_raps_gen = [], [], []
psnr_gen_list, ssim_gen_list = [], []
phase_mse_list = []
flux_gt_list, flux_gen_list = [], []
all_metrics = []
```

```
a = 0.1
arcsinh_1_over_a = np.arcsinh(1.0 / a)
```

```
for i in range(num_images):
    img_in = inputs[i, 0]
    img_gt = targets[i, 0]
    img_gen = generated[i, 0]
```

```
# 頻域與結構指標
```

```
all_raps_in.append(calculate_raps_fast(img_in))
all_raps_gt.append(calculate_raps_fast(img_gt))
all_raps_gen.append(calculate_raps_fast(img_gen))
```

```
psnr_val = psnr(img_gt, img_gen, data_range=2.0)
ssim_val = ssim(img_gt, img_gen, data_range=2.0)
psnr_gen_list.append(psnr_val)
ssim_gen_list.append(ssim_val)
```

```
# 相位誤差
```

```
phase_gt = calculate_phase(img_gt)
phase_gen = calculate_phase(img_gen)
```

```

phase_diff = np.angle(np.exp(1j * (phase_gt - phase_gen)))
phase_mse = np.mean(phase_diff**2)
phase_mse_list.append(phase_mse)

# 線性物理空間光通量
gt_01 = np.clip((img_gt * 0.55) + 0.5, 0.0, 1.0)
gen_01 = np.clip((img_gen * 0.55) + 0.5, 0.0, 1.0)
gt_linear = a * np.sinh(gt_01 * arcsinh_1_over_a)
gen_linear = a * np.sinh(gen_01 * arcsinh_1_over_a)

star_mask = gt_linear > 1e-4
flux_gt = np.sum(gt_linear[star_mask])
flux_gen = np.sum(gen_linear[star_mask])

flux_gt_list.append(flux_gt)
flux_gen_list.append(flux_gen)

# 紀錄單張數據
flux_error = np.abs(flux_gt - flux_gen) / flux_gt * 100 if flux_gt > 1e-5 else np.nan
all_metrics.append({
    "Image_ID": f"Test_Sample_{i:02d}",
    "PSNR (dB)": round(psnr_val, 4),
    "SSIM": round(ssim_val, 4),
    "Phase_MSE (rad)": round(phase_mse, 4),
    "GT_Linear_Flux": round(flux_gt, 2),
    "Gen_Linear_Flux": round(flux_gen, 2),
    "Linear_Flux_Error (%)": round(flux_error, 2)
})

# =====
# 3. 視覺化報表與輸出
# =====
mean_raps_in = np.mean(all_raps_in, axis=0)
mean_raps_gt = np.mean(all_raps_gt, axis=0)
mean_raps_gen = np.mean(all_raps_gen, axis=0)
std_raps_gt = np.std(all_raps_gt, axis=0)
std_raps_gen = np.std(all_raps_gen, axis=0)
freq_len = len(mean_raps_gt) // 2
x_freq = np.arange(freq_len)

fig = plt.figure(figsize=(18, 12))

# 顯示樣本比較

```

```

for idx in range(3):
    ax = plt.subplot(3, 3, idx+1)
    imgs = [inputs, targets, generated]
    titles = ["Input", "Ground Truth", "Output"]
    ax.imshow(imgs[idx][0, 0], cmap='magma', origin='lower', vmin=-1, vmax=1)
    ax.set_title(titles[idx])
    ax.axis('off')

# 頻域 RAPS
ax_raps = plt.subplot(3, 1, 2)
ax_raps.plot(x_freq, mean_raps_gt[:freq_len], label='Ground Truth')
ax_raps.plot(x_freq, mean_raps_gen[:freq_len], label='Output')
ax_raps.set_yscale('log')
ax_raps.legend()
ax_raps.set_title("Mean RAPS Analysis")

# 物理通量散佈圖
flux_gt_arr, flux_gen_arr = np.array(flux_gt_list), np.array(flux_gen_list)
ax_flux = plt.subplot(3, 1, 3)
ax_flux.scatter(flux_gt_arr, flux_gen_arr, alpha=0.7)
ax_flux.plot([flux_gt_arr.min(), flux_gt_arr.max()], [flux_gt_arr.min(), flux_gt_arr.max()], 'r-')
ax_flux.set_title("Physical Flux Conservation")

plt.tight_layout()
plt.show()

# 匯出 CSV 報表
df_metrics = pd.DataFrame(all_metrics)
output_csv_path = "PI_cGAN_V4_Finetune_results/picgan_v4_metrics_report.csv"
df_metrics.to_csv(output_csv_path, index=False, encoding='utf-8-sig')
print(f"報表已匯出至: {output_csv_path}")
display(df_metrics.tail())

```